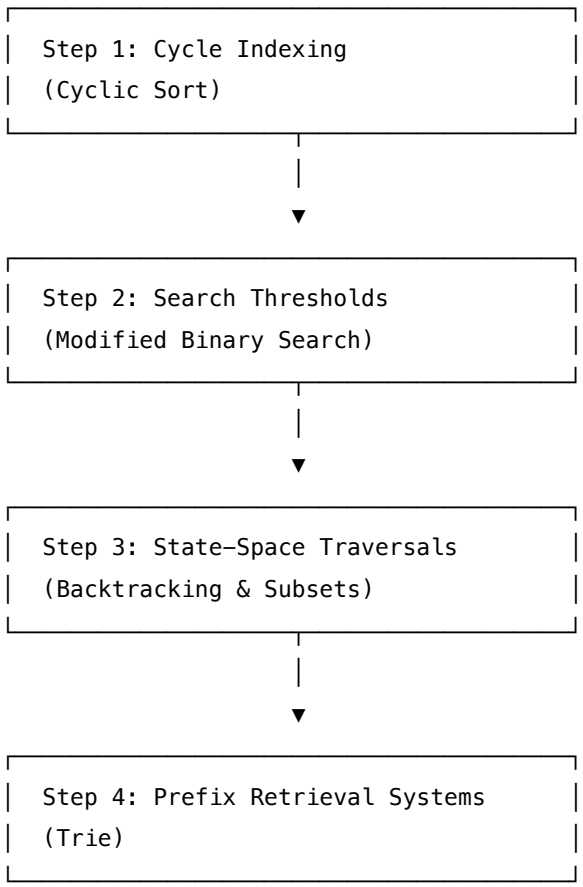
Contents

1 Tier 3: Advanced — Coding Interview Preparation Roadmap

Welcome to the **Advanced (Tier 3) Coding Roadmap**. This directory serves as your structured study guide and indexed reference repository for advanced algorithmic design. These patterns cover complex in-place cycle sorting, search space bisections, NP-complete recursive state pruning, and efficient prefix tree search systems.

1.1 Progression Roadmap

To build advanced algorithmic intuition, start with linear index-cycle logic, move to threshold binary searches, then explore backtracking state-spaces and prefix structures.



1.2 Detailed Learning Modules & File Index

Here is the complete catalog of the 19 Advanced problems, categorized by pattern, and arranged in recommended learning order.

1.2.1 Module 1: Cyclic Sort (In-Place Index Matching)

Goal: Sort arrays containing values in the range $[0, n]$ *in* $\mathcal{O}(n)$ *time using* $\mathcal{O}(1)$ *auxiliary space by swap-positioning.*

Pattern	Problem Name	Source	Difficulty	Core Concepts
Cyclic Sort	Missing Number	LeetCode 268	Easy	Value-to-index swap checks, XOR alternative

Pattern	Problem Name	Source	Difficulty	Core Concepts
Cyclic Sort	Find All Duplicates in Array	LeetCode 442	Medium	Index-negation marker flag, in-place cycle sort
Cyclic Sort	Find All Numbers Disappeared	LeetCode 448	Easy	Value sign-negation state mapping, swap cycle

1.2.2 Module 2: Modified Binary Search

Goal: Divide search spaces over non-standard ranges, rotated arrays, and numeric threshold bounds.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Modified Binary Search	Search in Rotated Sorted Array	LeetCode 33	Medium	Rotated partition identification, boundary narrowing
Modified Binary Search	Split Array Largest Sum	LeetCode 410	Hard	Feasibility test function, search space bisection
Modified Binary Search	Koko Eating Bananas	LeetCode 875	Medium	Monotonic rate search space, ceiling divisions

1.2.3 Module 3: Backtracking & Subsets

Goal: Traverse permutations, combinations, and partitioned state-spaces using recursion, backtracks, and early structural pruning.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Backtracking	Letter Combinations of a Phone Number	LeetCode 17	Medium	String-mapping array backtracking, BFS queue
Backtracking	Combination Sum	LeetCode 39	Medium	Array sorting, backtracking pruning, index preservation
Backtracking	Subsets	LeetCode 78	Medium	Cascading subsets build, bitmask integer indices

Pattern	Problem Name	Source	Difficulty	Core Concepts
Backtracking	Matchsticks to Square	LeetCode 473	Medium	Descending sorting optimization, symmetry breaks
Backtracking	Zuma Game	LeetCode 488	Hard	String board collapsing, DFS memoization, heuristic pruning
Backtracking	Non-decreasing Subsequences	LeetCode 491	Medium	Subsequence order preservation, index duplicate skipping
Subsets	Permutations	LeetCode 46	Medium	In-place swapping backtracking, std::next_permutation
Subsets	Combinations	LeetCode 77	Medium	Size boundary limits pruning, Gosper's Hack
Subsets	Subsets II	LeetCode 90	Medium	Array sorting, recursion-level duplicate skipping

1.2.4 Module 4: Trie (Prefix Trees)

Goal: Design and build efficient character prefix retrieval systems for spelling lookups, wildcard searches, and grid board validations.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Trie	Implement Trie (Prefix Tree)	LeetCode 208	Medium	Alphabet child-pointer nodes, insertion, prefix searches
Trie	Design Add and Search Words	LeetCode 211	Medium	Dot wildcard recursive character tree search
Trie	Word Search II	LeetCode 212	Hard	Matrix floodfill combined with Trie search prefix pruning
Trie	Concatenated Words	LeetCode 472	Hard	DP string word-break partitioning with Trie lookups

---

1.3 Advanced Tier Interview Strategy

1.3.1 1. Cyclic Sort index mappings

- **Loop Invariants**: The key invariant of cyclic sort is that at each index `i`, we repeatedly swap the element `nums[i]` with the element at its target index `nums[nums[i] - 1]` until the value at `i` is in its correct place (i.e. `nums[i] == i + 1` or `nums[i]` is out of bounds).
- **Infinite Loop Safety**: Always verify that the swap target index does not contain the same value as the source (`nums[i] != nums[nums[i] - 1]`). If they are equal, do not swap, otherwise your loop will run infinitely.

1.3.2 2. Backtracking Pruning for NP-Complete Problems

- **Descending Sorting**: In partitioning problems (such as Matchsticks to Square), sorting the input array in descending order before running backtracking allows the algorithm to try larger values first. This triggers early failures on large items and prunes search branches much faster.
- **Symmetry Breaking**: If multiple sub-buckets/sides are identical (e.g. side sums), and a search step fails for one side, do not try it on other identical sides with the same sum. Immediately exit or skip.

1.3.3 3. Trie Memory Management

- **C++ RAII Destructors**: Trie nodes contain heap-allocated child pointers. To prevent memory leaks, you must write a recursive destructor in C++:

```
~TrieNode() {
    for (TrieNode* child : children) {
        delete child;
    }
}
```

- **Python Memory Savings**: Trie node allocations can create millions of small objects, which bloats Python's memory due to instance `__dict__` overhead. Declare `__slots__ = ('children', 'is_end')` in your node classes to restrict dynamic attributes and reduce heap space by up to **70%**.

2 Missing Number

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 268, Glassdoor
- **Flashcards**: Cyclic Sort deck

2.1 Question Description

Given an array `nums` containing n distinct numbers in the range $[0, n]$, return *the only number in the range that is missing from the array.*

2.1.1 Examples

- **Input**: `nums = [3,0,1]`
  - **Output**: `2`
  - **Explanation**: $n = 3$ since there are 3 numbers, so all numbers are in the range $[0, 3]$. `2` is the missing number in the range since it does not appear in `nums`.
- **Input**: `nums = [0,1]`
  - **Output**: `2`
  - **Explanation**: $n = 2$ since there are 2 numbers, so all numbers are in the range $[0, 2]$. `2` is the missing number in the range since it does not appear in `nums`.
- **Input**: `nums = [9,6,4,2,3,5,7,0,1]`
  - **Output**: `8`
  - **Explanation**: $n = 9$ since there are 9 numbers, so all numbers are in the range $[0, 9]$. `8` is the missing number in the range since it does not appear in `nums`.

2.2 Detailed Solution (C++)

The core algorithm is **Cyclic Sort**. The key idea is that since the array contains numbers in the range $[0, n]$, each number v should ideally be placed at index v (except for n, which has no valid index in a 0-indexed array of size n).

We iterate through the array: 1. If the current number `nums[i]` is less than n and is not at its correct index (i.e., `nums[i] != nums[nums[i]]`), we swap `nums[i]` with the element at its correct index `nums[nums[i]]`. 2. We repeat the check for the new element swapped into `nums[i]` without advancing the loop index i. 3. If `nums[i] >= n` or is already at its correct index, we advance i.

After placing all possible elements in their correct indices, we make a final linear pass: the first index i where `nums[i] != i` represents the missing number. If all numbers from 0 to $n - 1$ are present at their matching indices, then the missing number must be n.

2.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <utility>

class Solution {
public:
    int missingNumber(std::vector<int>& nums) noexcept {
        int n = static_cast<int>(nums.size());
        int i = 0;
```

```
        // Phase 1: Cyclic Sort in-place
        while (i < n) {
            int correct = nums[i];
            // Swap to correct position if nums[i] is in the range [0, n - 1]
            // and it is not already at its correct position
            if (correct < n && nums[i] != nums[correct]) {
                std::swap(nums[i], nums[correct]);
            } else {
                i++;
            }
        }

        // Phase 2: Find the missing element
        for (i = 0; i < n; ++i) {
            if (nums[i] != i) {
                return i;
            }
        }

        return n;
    }
};
```

2.3 Detailed Solution (Python)

2.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using Cyclic Sort.

```
from typing import List

class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        """
        Finds the missing number in the range [0, n] using Cyclic Sort.

        Time Complexity: O(n)
        Space Complexity: O(1)
        """
        i = 0
        n = len(nums)

        # Phase 1: Cyclic Sort
        while i < n:
```

```
            correct = nums[i]
            # Verify correct index is within boundary and the value is not already sorted
            if correct < n and nums[i] != nums[correct]:
                nums[i], nums[correct] = nums[correct], nums[i]
            else:
                i += 1

        # Phase 2: Identify mismatch
        for i in range(n):
            if nums[i] != i:
                return i

        return n
```

2.4 Python-Specific Complexities & Implementation Considerations

2.4.1 1. The Trap of Dynamic Index Reassignment

- A common Python gotcha is trying to swap in a single line using: `nums[i], nums[nums[i]] = nums[nums[i]], nums[i]`.
- In Python, the expressions on the right-hand side are evaluated first, but the assignments on the left-hand side are done sequentially from left to right. Because `nums[i]` is updated in the first assignment, the second target index `nums[i]` changes, leading to an incorrect swap or infinite loop.
- *Solution*: Always store the target index in a local variable (e.g., `correct = nums[i]`) before executing the swap, or use a temporary variable.

2.4.2 2. Space Optimization & Garbage Collection Overhead

- Using built-in Python constructs like `set(nums)` or sorting with `nums.sort()` violates the strict $\mathcal{O}(1)$ auxiliary space constraint.
- `set(nums)` creates a hash table under the hood, allocating memory proportional to the size of the array. The cyclic sort implementation operates purely in-place, preventing dynamic allocations and reducing memory pressure on Python's garbage collector.

2.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Each element is swapped at most once to its correct index. The loop index is incremented at most n times. The final linear scan takes $\mathcal{O}(n)$ time.

Metric	Complexity	Description
Space Complexity	$\mathcal{O}(1)$	The sort is performed completely in-place, requiring constant auxiliary storage.

2.6 Common Follow-Up Questions & How to Answer

2.6.1 Q1: How do you solve this problem if the array is read-only (immutable)?

- **The Problem**: Cyclic sort requires modifying the input array in-place. If the input is immutable, cyclic sort cannot be applied.
- **The Solutions**:
  1. **Math Formula**: Calculate the expected sum of the numbers $[0, n]$ using $S = \frac{n(n+1)}{2}$. Subtract the sum of the array from S to obtain the missing number.

```python
def missingNumberMath(nums: List[int]) -> int:
    n = len(nums)
    return (n * (n + 1)) // 2 - sum(nums)
```

  2. **Bitwise XOR**: XOR all indices and elements. Because $x \oplus x = 0$, every present element cancels out its index counterpart, leaving only the missing number.

```python
def missingNumberXOR(nums: List[int]) -> int:
    missing = len(nums)
    for i, num in enumerate(nums):
        missing ^= i ^ num
    return missing
```

2.6.2 Q2: What are the trade-offs between the Math and XOR solutions?

- **Integer Overflow**: In low-level languages (C++ or Java), if n is very large (e.g., $n \approx 2^{31} - 1$), the term $n(n+1)$ will overflow a standard 32-bit signed or unsigned integer. The XOR solution does not suffer from overflow since bitwise operations do not accumulate magnitude.
- **Hardware Efficiency**: XOR operations execute in a single CPU cycle on modern hardware and do not require multiplication or division, making it highly optimal at the machine level.

2.7 Pro-Tip: How to Impress the Interviewer

- **Mention Cache Line Thrashing**: Highlight that cyclic sort performs arbitrary memory swaps across the array. While it achieves optimal theoretical time complexity, it can cause cache misses and line thrashing on large datasets. If memory is not constrained, a linear-scan math approach has much better spatial locality and will run faster on modern CPUs.
- **Avoid the Unsigned Underflow on Empty Checks**: In C++, always be mindful of unsigned arithmetic when working with `std::vector::size()`. Even though this specific problem guarantees $n \geq 1$, explicitly casting `size()` to a signed integer (`int`) shows defensive programming hygiene.

3 Find All Duplicates in an Array

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 442, Glassdoor
- **Flashcards**: Cyclic Sort deck

3.1 Question Description

Given an integer array `nums` of length n where all the integers of `nums` are in the range $[1, n]$ and each integer appears **at most twice**, return *an array of all the integers that appear twice.*

You must write an algorithm that runs in $\mathcal{O}(n)$ time and uses only constant auxiliary space, excluding the space needed to store the output.

3.1.1 Examples

- **Input**: `nums = [4,3,2,7,8,2,3,1]`
  - **Output**: `[2,3]`
- **Input**: `nums = [1,1,2]`
  - **Output**: `[1]`
- **Input**: `nums = [1]`
  - **Output**: `[]`

3.2 Detailed Solution (C++)

There are two main ways to solve this problem in-place using constant auxiliary space: **Index Negation** and **Cyclic Sort**.

3.2.1 Method 1: Index Negation (Sign Flips)

Since all numbers in the array are in the range $[1, n]$, each number can be mapped to a valid index in the array: $idx = |num| - 1$. 1. We iterate through the array, and for each number `num`, we look at the element at index $idx = |num| - 1$. 2. If `nums[idx]` is positive, we negate it (`nums[idx] = -nums[idx]`) to mark that we have seen the number $idx + 1$. 3. If `nums[idx]` is already negative, it means we have visited this index before. Thus, $|num|$ must be a duplicate, and we add it to our result list.

3.2.2 Method 2: Cyclic Sort

We try to place each number x at its correct index $x - 1$. 1. While `nums[i]` is not at its correct position (i.e. `nums[i] != nums[nums[i] - 1]`), we swap `nums[i]` with the element at index `nums[i] - 1`. 2. If the element at the target index is already correct, we stop swapping to avoid an infinite loop and increment i. 3. After sorting, we iterate through the array: any element where `nums[i] != i + 1` is a duplicate.

3.2.3 Standard C++ Production Code

The following implementation uses the **Index Negation** technique, which is extremely compact and avoids swaps.

```cpp
#include <vector>
#include <cmath>
#include <cstdlib>

class Solution {
public:
    std::vector<int> findDuplicates(std::vector<int>& nums) {
        std::vector<int> result;

        // Optimizing allocation: pre-allocate memory to avoid multiple reallocations
        result.reserve(nums.size() / 2);

        for (int num : nums) {
            int idx = std::abs(num) - 1;

            if (nums[idx] < 0) {
                // Already visited, meaning this number is a duplicate
                result.push_back(std::abs(num));
            } else {
                // Mark as visited by negating
                nums[idx] = -nums[idx];
            }
        }

        return result;
    }
};
```

3.3 Detailed Solution (Python)

3.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using the Index Negation method.

```python
from typing import List

class Solution:
    def findDuplicates(self, nums: List[int]) -> List[int]:
        """
        Finds all duplicate numbers in-place.
```

```
    Time Complexity: O(n)
    Space Complexity: O(1) auxiliary
    """
    result = []

    for num in nums:
        idx = abs(num) - 1
        if nums[idx] < 0:
            result.append(abs(num))
        else:
            nums[idx] = -nums[idx]

    return result
```

3.4 Python-Specific Complexities & Implementation Considerations

3.4.1 1. In-Place Mutation Side Effects

- Python variables are references. When you pass `nums` to `findDuplicates`, the function modifies the original list in place.
- If the caller expects the input list to remain unchanged, mutating the array in-place is a side effect. It is a good practice during interviews to ask the interviewer: *"Is it acceptable to modify the input list, or should I restore it before returning?"* (See follow-up questions below for how to restore the array).

3.4.2 2. Built-in `abs()` Performance

- Using Python's built-in `abs()` is highly efficient because it is implemented in C at the interpreter level. It is faster than manual ternary-based signs checks.

3.4.3 3. List Comprehension and Auxiliary Space

- We could attempt to build the result list using a comprehension or filters, but doing so without mutating `nums` requires tracking seen numbers in a `set` or auxiliary list, raising the space complexity to $\mathcal{O}(n)$. Modifying the list in-place remains the only way to satisfy the $\mathcal{O}(1)$ space requirement.

3.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We perform a single linear scan of the array. All lookups and updates at `nums[idx]` are $\mathcal{O}(1)$.
Space Complexity	$\mathcal{O}(1)$	The space complexity is $\mathcal{O}(1)$ auxiliary since we mutate the input array in-place. The output list is excluded from the auxiliary space complexity per the problem constraints.

3.6 Common Follow-Up Questions & How to Answer

3.6.1 Q1: How do you solve this using Cyclic Sort instead of Index Negation?

- **The Solution**: Place each number x at its correct index $x - 1$ using swaps.
- **C++ Code**:

```cpp
std::vector<int> findDuplicatesCyclic(std::vector<int>& nums) {
    int n = nums.size();
    int i = 0;
    while (i < n) {
        int correct = nums[i] - 1;
        if (nums[i] != nums[correct]) {
            std::swap(nums[i], nums[correct]);
        } else {
            i++;
        }
    }
    std::vector<int> result;
    for (i = 0; i < n; i++) {
        if (nums[i] != i + 1) {
            result.push_back(nums[i]);
        }
    }
    return result;
}
```

3.6.2 Q2: What if we are required to restore the input array to its original state before returning?

- **The Solution**: We can perform a second pass to restore all negative values back to positive.
- **Python Code**:

```python
def findDuplicatesAndRestore(nums: List[int]) -> List[int]:
    result = []
    for num in nums:
        idx = abs(num) - 1
        if nums[idx] < 0:
            result.append(abs(num))
        else:
            nums[idx] = -nums[idx]

    # Restore the array
    for i in range(len(nums)):
        if nums[i] < 0:
            nums[i] = -nums[i]
    return result
```

This maintains $\mathcal{O}(n)$ time and $\mathcal{O}(1)$ auxiliary space.

3.6.3 Q3: What if numbers can appear more than twice?

- **The Problem**: If a number can appear three or more times, Index Negation will fail because the third occurrence will see a negative number and flip it back to positive (or misidentify it as a duplicate again).
- **The Solution**: Using Cyclic Sort is robust here. If we find that `nums[i]` is not in its correct position but `nums[nums[i] - 1]` already contains the correct value, we simply advance `i`. In a final pass, any element `nums[i]` where `nums[i] != i + 1` is a duplicate. We can deduplicate the results using a hash set or checking if the duplicate has already been added.

3.7 Pro-Tip: How to Impress the Interviewer

- **Design for API Safety**: Highlight that mutating inputs without restoring them violates the principle of least surprise in API design. Showing the interviewer that you know how to restore the array in $\mathcal{O}(n)$ time demonstrates production-grade thinking.
- **Prevent Vector Reallocations**: In C++, `std::vector` resizes dynamically, which can double its capacity and copy elements during execution. Reserving memory upfront using `result.reserve(nums.size() / 2)` prevents costly allocations and copies.
- **Mention Cache Performance**: Cyclic sort requires multiple random memory writes (swaps), which can lead to high L1/L2 cache miss rates. The Index Negation approach reads the array sequentially and writes only once per index, making it significantly more cache-friendly and faster in practice.

4 Find All Numbers Disappeared in an Array

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect

- **Source**: LeetCode 448, Glassdoor
- **Flashcards**: Cyclic Sort deck

4.1 Question Description

Given an array `nums` of n integers where `nums[i]` is in the range $[1, n]$, return *an array of all the integers in the range* $[1, n]$ *that do not appear in* `nums`.

4.1.1 Examples

- **Input**: `nums = [4,3,2,7,8,2,3,1]`
  - **Output**: `[5,6]`
- **Input**: `nums = [1,1]`
  - **Output**: `[2]`

4.2 Detailed Solution (C++)

Just like LeetCode 442, this problem restricts numbers to the range $[1, n]$, which allows us to map each number to a valid index $idx = |num| - 1$. We can solve it in-place using two different strategies: **Index Negation** and **Cyclic Sort**.

4.2.1 Method 1: Index Negation (Sign Flips)

1. Iterate through the array. For each value `num`, calculate the target index $idx = |num| - 1$.
2. If `nums[idx]` is positive, flip its sign to negative (`nums[idx] = -nums[idx]`). This serves as a status flag indicating that the number $idx + 1$ exists in the array. If the number is already negative, do nothing.
3. Perform a second pass. For any index i where `nums[i]` is positive, the number $i + 1$ was never seen. Append $i + 1$ to the results.

4.2.2 Method 2: Cyclic Sort

1. Iterate through the array and try to place each number `nums[i]` at its correct index `nums[i] - 1` by swapping.
2. Advance the loop only when `nums[i]` is in the correct spot or if `nums[i] == nums[nums[i]-1]`.
3. In a second pass, any index i where `nums[i] != i + 1` means the number $i + 1$ is missing.

4.2.3 Standard C++ Production Code

The following implementation uses the **Index Negation** technique.

```cpp
#include <vector>
#include <cmath>
#include <cstdlib>
```

```cpp
class Solution {
public:
    std::vector<int> findDisappearedNumbers(std::vector<int>& nums) {
        int n = static_cast<int>(nums.size());

        // Phase 1: Mark present numbers by negating elements at corresponding indices
        for (int num : nums) {
            int index = std::abs(num) - 1;
            if (nums[index] > 0) {
                nums[index] = -nums[index];
            }
        }

        // Phase 2: Find all indices that remain positive
        std::vector<int> result;
        for (int i = 0; i < n; ++i) {
            if (nums[i] > 0) {
                result.push_back(i + 1);
            }
        }

        return result;
    }
};
```

4.3 Detailed Solution (Python)

4.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using the Index Negation method.

```python
from typing import List

class Solution:
    def findDisappearedNumbers(self, nums: List[int]) -> List[int]:
        """
        Finds all missing numbers in the range [1, n] in O(n) time and O(1) auxiliary space.

        Time Complexity: O(n)
        Space Complexity: O(1) auxiliary
        """
        # Phase 1: Mark visited indices
        for num in nums:
            index = abs(num) - 1
```

```
        if nums[index] > 0:
            nums[index] = -nums[index]

    # Phase 2: Identify positive values and yield the missing numbers
    return [i + 1 for i in range(len(nums)) if nums[i] > 0]
```

4.4 Python-Specific Complexities & Implementation Considerations

4.4.1 1. High-Performance List Comprehension

- The Python implementation uses a list comprehension `[i + 1 for i in range(len(nums)) if nums[i] > 0]` to construct the final list. List comprehensions run at C-speed in CPython and are faster than executing a manual `for` loop with `list.append()`.

4.4.2 2. Side-Effect Safety & Array Restoration

- Modifying `nums` in-place alters the caller's array. If the interviewer requests a side-effect-free function while maintaining $\mathcal{O}(1)$ auxiliary space, we must restore the list before returning.
- *Solution*: Perform a restorative pass before returning.

```
missing = [i + 1 for i in range(len(nums)) if nums[i] > 0]
for i in range(len(nums)):
    if nums[i] < 0:
        nums[i] = -nums[i]
return missing
```

4.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We perform two sequential linear scans of the array. The runtime is linear.
Space Complexity	$\mathcal{O}(1)$	Auxiliary space is $\mathcal{O}(1)$ as the input array is modified in-place and the output list does not count toward the space limit.

4.6 Common Follow-Up Questions & How to Answer

4.6.1 Q1: How do you implement this using Cyclic Sort?

- **The Solution**: Place each number at its target index using swaps.

- **Python Code**:

```python
def findDisappearedNumbersCyclic(nums: List[int]) -> List[int]:
    i = 0
    n = len(nums)
    while i < n:
        correct = nums[i] - 1
        if nums[i] != nums[correct]:
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    return [i + 1 for i in range(n) if nums[i] != i + 1]
```

4.6.2 Q2: What if the input array is completely read-only?

- **The Problem**: If the array is read-only, in-place negation and cyclic sort are prohibited.
- **The Solution**: We must use auxiliary memory.
  1. **Hash Set**: Store all elements in a set, then scan $[1, n]$ to find elements not in the set. Time: $\mathcal{O}(n)$, Space: $\mathcal{O}(n)$.
  2. **Bitset / Boolean Array**: If n is large, we can use a bit array (e.g. `std::vector<bool>` in C++) to save memory (takes $\frac{n}{8}$ bytes instead of $4n$ bytes).

4.7 Pro-Tip: How to Impress the Interviewer

- **Highlight Write Amplification and Pipeline Stalls**: Compare Index Negation to Cyclic Sort. Negation only writes to memory once per element (if not already negative) and does not perform swaps. Cyclic sort executes random swaps which trigger memory writes and register operations, leading to higher **write amplification** and CPU cache invalidations in multi-core/concurrent settings.
- **Vector Reallocation Avoidance**: Explicitly note that while we don't know the exact size of the output array, in C++ it is usually better to return the result by value. Modern C++ compilers will use **Return Value Optimization (RVO)** or move semantics to return the `std::vector` with zero copy overhead.

5 Search in Rotated Sorted Array

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 33, Glassdoor
- **Flashcards**: Modified Binary Search deck

5.1 Question Description

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is possibly rotated at an unknown pivot index `k` ($1 \leq k <$ nums.length) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be rotated at pivot index `3` and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`.

You must write an algorithm with $\mathcal{O}(\log n)$ runtime complexity.

5.1.1 Examples

- **Input**: `nums = [4,5,6,7,0,1,2]`, `target = 0`
  - **Output**: `4`
- **Input**: `nums = [4,5,6,7,0,1,2]`, `target = 3`
  - **Output**: `-1`
- **Input**: `nums = [1]`, `target = 0`
  - **Output**: `-1`

5.2 Detailed Solution (C++)

The core algorithm is a modified **Binary Search**. In a rotated sorted array, splitting the array in half guarantees that at least one of the halves is normally sorted. We can identify which half is sorted by comparing the boundary values: 1. If `nums[left] <= nums[mid]`, the left half is sorted. * If the target lies within [nums[$left$], nums[mid]), we narrow our search to the left half. * Otherwise, we search the right half. 2. If `nums[mid] < nums[left]`, the right half must be sorted. * If the target lies within (nums[mid], nums[$right$]], we search the right half. * Otherwise, we search the left half.

5.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <cstddef>

class Solution {
public:
    int search(const std::vector<int>& nums, int target) noexcept {
        // Edge Case: Handle empty collection explicitly to prevent unsigned underflow
        if (nums.empty()) {
            return -1;
        }

        int left = 0;
```

```
        int right = static_cast<int>(nums.size()) - 1; // Safely cast size to int

        while (left <= right) {
            int mid = left + (right - left) / 2; // Prevent integer overflow (left + right) / 2

            if (nums[mid] == target) {
                return mid;
            }

            // Determine which half is normally sorted
            if (nums[left] <= nums[mid]) {
                // Left side is sorted
                if (target >= nums[left] && target < nums[mid]) {
                    right = mid - 1; // Target is in the left sorted portion
                } else {
                    left = mid + 1;  // Target is in the right portion
                }
            } else {
                // Right side is sorted
                if (target > nums[mid] && target <= nums[right]) {
                    left = mid + 1;  // Target is in the right sorted portion
                } else {
                    right = mid - 1; // Target is in the left portion
                }
            }
        }

        return -1; // Target not found
    }
};
```

5.3 Detailed Solution (Python)

5.3.1 Standard Python Production Code

Below is the iterative, type-hinted Python implementation. It avoids recursive call overhead and ensures true constant space.

```
from typing import List

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        """
        Searches for a target in a rotated sorted array using binary search.
```

```
    Time Complexity: O(log N)
    Space Complexity: O(1)
    """
    # Edge Case: Handle empty collection explicitly
    if not nums:
        return -1

    left, right = 0, len(nums) - 1

    while left <= right:
        # Floor division operator '//' is required in Python to get integer index.
        # 'left + (right - left) // 2' is used to demonstrate system-level safety,
        # preventing overflow in languages with fixed-width integers.
        mid = left + (right - left) // 2

        if nums[mid] == target:
            return mid

        # Determine which half is normally sorted
        if nums[left] <= nums[mid]:
            # Left side is sorted
            if nums[left] <= target < nums[mid]:
                right = mid - 1  # Target is within the sorted left portion
            else:
                left = mid + 1   # Search the right portion
        else:
            # Right side is sorted
            if nums[mid] < target <= nums[right]:
                left = mid + 1   # Target is within the sorted right portion
            else:
                right = mid - 1  # Search the left portion

    return -1
```

5.4 Python-Specific Complexities & Implementation Considerations

5.4.1 1. Integer Division: `/` vs `//`

- In Python 3, the single slash division operator `/` returns a floating-point number (e.g., `5 / 2 = 2.5`). Python list indexes must be integers; using a float results in a `TypeError`.
- Always use the floor division operator `//` (e.g., `5 // 2 = 2`) to ensure index variables remain integers.

5.4.2 2. Arbitrary-Precision Integers vs System Limits

- Python handles arbitrary-precision integers natively (it automatically transitions to "large integers" when values exceed $2^{63}-1$). Thus, `(left + right) // 2` cannot overflow in Python.
- However, writing `left + (right - left) // 2` is still recommended in system programming interviews. It signals that you write portable, compiler-agnostic code that translates safely to low-level environments (C/C++, Rust, CUDA).

5.4.3 3. List Slicing Performance Penalty

- A common Python mistake is to slice arrays during search (e.g., `search(nums[:mid])`). Slicing a list in Python creates a **shallow copy** of that slice, taking $\mathcal{O}(k)$ time and space (where k is the slice length).
- Doing this degrades the algorithm's time complexity from $\mathcal{O}(\log n)$ to $\mathcal{O}(n)$, and space from $\mathcal{O}(1)$ to $\mathcal{O}(n)$. Always use pointer boundaries (`left`, `right`) to partition the search space in-place.

5.4.4 4. Recursion Limit and Frame Allocations

- If binary search is implemented recursively, each call allocates a new stack frame in Python. Because Python's default call stack limit is low (typically 1000), deep recursive searches can cause a `RecursionError` on large datasets.
- Additionally, frame allocation in Python has higher CPU overhead compared to C++ because of dynamic object bindings. The iterative implementation is always preferred for optimal performance.

5.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(\log n)$	The search space is halved in each step of the binary search loop.
Space Complexity	$\mathcal{O}(1)$	Only a few integer variables are maintained, requiring constant auxiliary memory.

5.6 Common Follow-Up Questions & How to Answer

5.6.1 Q1: What if duplicates are allowed in the array? (LeetCode 81)

- **The Problem**: If the array has duplicate values (e.g., `nums = [1, 1, 1, 1, 0, 1, 1]`), we cannot determine if the left or right side is sorted when `nums[left] == nums[mid] == nums[right]`.
- **The Solution**: When this edge case is hit, we must shrink both boundaries (`left++` and `right--`) until they differ.

- **Code Addition**:

```
if (nums[left] == nums[mid] && nums[mid] == nums[right]) {
    left++;
    right--;
    continue;
}
```

- **Complexity Impact**: The worst-case time complexity degrades to $\mathcal{O}(n)$ (when all elements are duplicates and we must scan linearly), though average-case remains $\mathcal{O}(\log n)$.

5.6.2 Q2: How can we implement this logic "branchlessly" to prevent CPU branch mispredictions?

- **The Problem**: Binary search is notoriously slow on CPUs due to **branch mispredictions** (the search direction has a 50% probability, which confuses the hardware branch predictor and flushes the execution pipeline).
- **The Solution**: We can eliminate branching by using data dependencies (ternary selectors or arithmetic) to compute the next boundaries instead of using `if/else` statements. Modern compilers translate ternary operators like `cond ? val1 : val2` into conditional move instructions (`CMOV` on x86, `SEL` on ARM/GPU), which execute without branch prediction.
- **Example (Conceptual Branchless Update)**:

```
// Instead of if-else:
int left_sorted = (nums[left] <= nums[mid]);
int target_in_left = (target >= nums[left]) & (target < nums[mid]);
int target_in_right = (target > nums[mid]) & (target <= nums[right]);

int choose_left = left_sorted ? target_in_left : !target_in_right;

// Update indices conditionally
right = choose_left ? (mid - 1) : right;
left = choose_left ? left : (mid + 1);
```

5.6.3 Q3: How would you parallelize this search on a GPU (CUDA) for multiple targets?

- **The Problem**: If 32 threads in a single **warp** search for different targets in parallel, their paths will diverge (some branch left, some right), causing **warp divergence** which serializes execution.
- **Optimizations**:
  1. **Shared Memory**: Store the array in GPU shared memory (`__shared__`) if it is accessed repeatedly by all threads. Shared memory has low latency and high bandwidth.
  2. **Memory Coalescing**: Ensure that targets queried by adjacent threads are stored contiguously in memory so the GPU can load them in a single memory transaction.
  3. **Bank Conflicts**: Ensure the array accesses do not fall into the same shared memory bank (modulo 32 addressing).

5.7 Pro-Tip: How to Impress the Interviewer

- **Catch the Underflow Bug Proactively**: Point out that doing `nums.size() - 1` without checking if `nums` is empty is a classic security and stability bug. Since `size()` returns `size_t` (unsigned), an empty vector yields `0 - 1 = 18446744073709551615` (on 64-bit platforms), causing an immediate out-of-bounds crash in the loop condition `left <= right`. Casting to `int` or explicitly handling empty checks proves real-world C++ systems hygiene.
- **Mention Cache Prefetching**: Mention that while binary search has $\mathcal{O}(\log n)$ complexity, for small arrays ($n \leq 64$), a simple linear scan (`std::find`) is often faster in practice due to contiguous hardware prefetching and the lack of branch mispredictions.
- **Highlight CPU Pipeline Stalls**: Discussing CPU pipelining and branch misprediction costs shows you think about how code interacts with the physical silicon, which is critical for low-level development at companies like NVIDIA.

6 Split Array Largest Sum

- **Category**: Coding
- **Difficulty**: Hard
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 410, Glassdoor
- **Flashcards**: Modified Binary Search deck

6.1 Question Description

Given an integer array `nums` and an integer `k`, split `nums` into `k` non-empty subarrays such that the largest sum of any subarray is minimized.

Return *the minimized largest sum of the split.*

A **subarray** is a contiguous part of the array.

6.1.1 Examples

- **Input**: `nums = [7,2,5,10,8]`, `k = 2`
  - **Output**: `18`
  - **Explanation**: There are four ways to split `nums` into two subarrays. The best way is to split it into `[7,2,5]` and `[10,8]`, where the largest sum among the two subarrays is only `18`.
- **Input**: `nums = [1,2,3,4,5]`, `k = 2`
  - **Output**: `9`
  - **Explanation**: The best way is to split it into `[1,2,3]` and `[4,5]`, where the largest sum among the two subarrays is only `9`.

6.2 Detailed Solution (C++)

The problem asks us to find a value S (the maximum subarray sum) such that we can partition the array into k contiguous subarrays where no subarray has a sum greater than S.

This is a classic example of **Binary Search on the Answer**.

1. **Define the search space `[left, right]`:**
   - `left` (minimum possible maximum sum): This must be at least the maximum single element in the array (`max(nums)`), because the element with the maximum value must belong to some subarray, and subarray sizes must be ≥ 1.
   - `right` (maximum possible maximum sum): This is the sum of all elements in the array (`sum(nums)`), which corresponds to the case where $k = 1$ (the entire array is a single subarray).
2. **Binary Search:**
   - Find the midpoint `mid = left + (right - left) / 2`.
   - Check if it is feasible to partition `nums` into $\leq k$ subarrays such that no subarray's sum exceeds `mid`.
3. **Feasibility Check (Greedy):**
   - Iterate through `nums` and accumulate elements into a `current` sum.
   - If `current` exceeds `mid`, we must start a new subarray. We increment our subarray count `count` and reset `current` to the current element.
   - If `count` exceeds `k`, then `mid` is too small, and we must increase the lower bound (`left = mid + 1`).
   - If the loop finishes within k subarrays, then `mid` is feasible, and we try to find a smaller maximum sum (`right = mid`).

6.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <numeric>
#include <algorithm>

class Solution {
private:
    // Helper function to check if a max sum is feasible with k splits
    bool feasible(const std::vector<int>& nums, long long max_sum, int k) noexcept {
        int count = 1;
        long long current = 0;

        for (int v : nums) {
            current += v;
            if (current > max_sum) {
                // Cannot fit in the current subarray, start a new one
                count++;
                current = v;
                if (count > k) {
```

```
                    return false; // Requires more than k subarrays
                }
            }
        }
        return true;
    }

public:
    int splitArray(const std::vector<int>& nums, int k) {
        if (nums.empty()) {
            return 0;
        }

        // Use long long to prevent integer overflow during accumulation
        long long left = *std::max_element(nums.begin(), nums.end());
        long long right = std::accumulate(nums.begin(), nums.end(), 0LL);

        while (left < right) {
            long long mid = left + (right - left) / 2;

            if (feasible(nums, mid, k)) {
                right = mid;      // Mid is feasible, try to find a smaller maximum sum
            } else {
                left = mid + 1;   // Mid is too small, increase target sum
            }
        }

        return static_cast<int>(left);
    }
};
```

6.3 Detailed Solution (Python)

6.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using Binary Search on the Answer.

```
from typing import List

class Solution:
    def splitArray(self, nums: List[int], k: int) -> int:
        """
        Finds the minimized largest sum of split subarrays using Binary Search.
```

```
    Time Complexity: O(n * log(S)) where S is the sum of nums.
    Space Complexity: O(1) auxiliary
    """
    # Establish the range boundaries
    left, right = max(nums), sum(nums)

    def feasible(max_sum: int) -> bool:
        count = 1
        current = 0
        for n in nums:
            current += n
            if current > max_sum:
                count += 1
                current = n
                if count > k:
                    return False
        return True

    # Binary search on range [max(nums), sum(nums)]
    while left < right:
        mid = left + (right - left) // 2
        if feasible(mid):
            right = mid      # mid is feasible, check for smaller solutions
        else:
            left = mid + 1   # mid is too small, increase minimum sum target

    return left
```

6.4 Python-Specific Complexities & Implementation Considerations

6.4.1 1. Closures vs Helper Methods

- In Python, nesting `feasible` inside `splitArray` creates a **closure**. This allows `feasible` to directly read `nums` and `k` from the parent stack frame without passing them as arguments.
- While closures are highly readable, lookups of variables in enclosing scopes in Python are slightly slower than local variable lookups. For performance-critical code or very large loops, passing the variables explicitly or refactoring to a class method can yield micro-optimizations.

6.4.2 2. Large Range Precision

- Python automatically handles arbitrary-precision integers, so variables like `right = sum(nums)` will never overflow.
- However, we still write `mid = left + (right - left) // 2` to emphasize code portability to low-level target environments.

6.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log S)$	The search space size is $S = \text{sum}(nums) - \max(nums)$. The binary search takes $\mathcal{O}(\log S)$ iterations. In each iteration, we scan the array of size n in $\mathcal{O}(n)$ time.
Space Complexity	$\mathcal{O}(1)$	We only maintain pointers and accumulators, using constant auxiliary space.

6.6 Common Follow-Up Questions & How to Answer

6.6.1 Q1: How can this problem be solved if elements can be negative?

- **The Problem**: If the array contains negative numbers, the prefix sum is no longer monotonically increasing. A greedy packing strategy in the feasibility check will fail (for example, adding an element might decrease the current subarray sum, which breaks the monotonic greedy assumption).
- **The Solution**: We must use **Dynamic Programming (DP)**.
  - Let $DP[i][j]$ be the minimized largest sum when splitting the first i elements into j subarrays.
  - The recurrence relations:

$$DP[i][j] = \min_{0 \le m < i} \left\{ \max(DP[m][j-1], \sum_{p=m+1}^{i} nums[p]) \right\}$$

  - **Complexity**: Time complexity becomes $\mathcal{O}(n^2 \cdot k)$, and space complexity is $\mathcal{O}(n \cdot k)$.

6.6.2 Q2: What are the similarities between this problem and LeetCode 1011 (Capacity To Ship Packages Within D Days)?

- **The Answer**: They are isomorphic. "Shipping packages within D days" maps exactly to "splitting the array into D subarrays" where the cargo weight is `nums` and we want to minimize the maximum daily weight. The exact same binary search on answer template solves both.

6.7 Pro-Tip: How to Impress the Interviewer

- **Define Monotonicity Formally**: Clarify that Binary Search on Answer is only possible because the feasibility function $f(S)$ is **monotonic**:

$$\text{If } f(S_0) = \text{True, then } f(S) = \text{True for all } S \geq S_0.$$

  This formal framing shows you understand the mathematical foundations of binary search beyond simple sorted array indices.
- **Identify Cache Friendliness**: Point out that the feasibility function performs a simple, contiguous linear scan of the input array. This means it benefits from 100% hardware L1/L2 cache prefetching, which executes extremely fast on modern CPUs, making it far superior to the Dynamic Programming solution even when k is large.

7 Koko Eating Bananas

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 875, Glassdoor
- **Flashcards**: Modified Binary Search deck

7.1 Question Description

Koko loves to eat bananas. There are n piles of bananas, the i^{th} pile has `piles[i]` bananas. The guards have gone and will come back in h hours.

Koko can decide her bananas-per-hour eating speed of k. Each hour, she chooses some pile of bananas and eats k bananas from that pile. If the pile has less than k bananas, she eats all of them instead and will not eat any more bananas during this hour.

Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return.

Return *the minimum integer k such that she can eat all the bananas within h hours.*

7.1.1 Examples

- **Input**: `piles = [3,6,7,11]`, `h = 8`
  - **Output**: `4`
- **Input**: `piles = [30,11,23,4,20]`, `h = 5`
  - **Output**: `30`
- **Input**: `piles = [30,11,23,4,20]`, `h = 6`
  - **Output**: `23`

7.2 Detailed Solution (C++)

This problem can be resolved using **Binary Search on the Answer**.

1. **Define the Search Space `[left, right]`**:
   - `left` (minimum possible speed): Koko must eat at least 1 banana per hour, so `left = 1`.
   - `right` (maximum possible speed): Koko can eat at most one pile per hour. Therefore, if she eats at a speed equal to the maximum pile size (`max(piles)`), she will finish each pile in exactly 1 hour. Any speed greater than `max(piles)` is redundant because she cannot eat from multiple piles within the same hour. Thus, `right = max(piles)`.
2. **Binary Search**:
   - We search in the range `[1, max(piles)]`.
   - For a candidate speed `mid`, we calculate the total hours needed to finish all piles:

$$\text{Hours} = \sum_{i=0}^{n-1} \lceil \text{piles}[i]/\text{mid} \rceil$$

   - If Hours $\leq h$, then `mid` is a valid speed. We try to find a slower speed by setting `right = mid`.
   - If Hours $> h$, Koko cannot finish in time. We must increase the speed by setting `left = mid + 1`.

7.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>

class Solution {
private:
    // Helper function to calculate total hours needed for eating speed k
    long long hoursNeeded(const std::vector<int>& piles, int k) noexcept {
        long long total = 0;
        for (int v : piles) {
            // Integer ceiling division: ceil(v / k) is equivalent to (v + k - 1) / k
            total += (static_cast<long long>(v) + k - 1) / k;
        }
        return total;
    }

public:
    int minEatingSpeed(const std::vector<int>& piles, int h) {
        int left = 1;
        int right = *std::max_element(piles.begin(), piles.end());

        while (left < right) {
            int mid = left + (right - left) / 2;
```

```
            if (hoursNeeded(piles, mid) <= h) {
                right = mid;      // mid is feasible, try to find a slower speed
            } else {
                left = mid + 1;   // too slow, must increase speed
            }
        }

        return left;
    }
};
```

7.3 Detailed Solution (Python)

7.3.1 Standard Python Production Code

Below is the type-hinted Python implementation. It utilizes integer division arithmetic to compute the ceiling values without floating-point precision issues.

```
from typing import List

class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        """
        Finds the minimum speed k to finish all bananas in h hours.

        Time Complexity: O(n * log(M)) where M is max(piles).
        Space Complexity: O(1) auxiliary
        """
        left, right = 1, max(piles)

        # Binary search on speed range
        while left < right:
            mid = left + (right - left) // 2

            # Integer-based ceiling division: (p + mid - 1) // mid
            # Sum the eating hours required for all piles at speed mid
            hours = sum((p + mid - 1) // mid for p in piles)

            if hours <= h:
                right = mid       # mid speed works, try to find a slower speed
            else:
                left = mid + 1    # Koko needs to eat faster

        return left
```

7.4 Python-Specific Complexities & Implementation Considerations

7.4.1 1. Integer Ceiling Division vs `math.ceil`

- In Python, calling `math.ceil(p / mid)` converts the division result into a floating-point number. Floats in Python are double-precision numbers (64-bit IEEE 754) which have 53 bits of precision. If piles contain elements near the upper limit (10^9) and eating speeds are small, floating-point divisions can theoretically run into precision limitations.
- Using the integer arithmetic trick `(p + mid - 1) // mid` is exact, fast, and does not require importing the `math` library.

7.4.2 2. Generator Expression vs List Comprehension

- The sum expression `sum((p + mid - 1) // mid for p in piles)` uses a generator expression, which does not allocate an intermediate list in memory (constant space).
- However, generators in CPython carry a small overhead for creating and iterating the generator frame. If n is small (e.g., $n \leq 10^4$), writing `sum([(p + mid - 1) // mid for p in piles])` (which creates a temporary list) or a simple linear `for` loop might run slightly faster, though it uses $\mathcal{O}(n)$ memory.

7.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log M)$	M is max(piles). The binary search takes $\mathcal{O}(\log M)$ steps. In each step, we iterate through the n piles to calculate the hours needed.
Space Complexity	$\mathcal{O}(1)$	Only a few integer boundaries and accumulators are maintained in memory.

7.6 Common Follow-Up Questions & How to Answer

7.6.1 Q1: What is the minimum possible value of h allowed by the constraints, and how does it affect the bounds?

- **The Answer**: The constraints state that piles.length $\leq h$. If $h =$ piles.length, then Koko has exactly 1 hour per pile. Thus, the eating speed k must be at least the maximum pile size max(piles) to ensure she finishes each pile within its single hour. The loop immediately exits and returns `max(piles)`.

7.6.2 Q2: How can we optimize this if h is extremely large (e.g., $h \geq \sum$ piles)?

- **The Answer**: If $h \geq \sum$ piles, Koko can eat at the minimum speed of 1 banana/hour and still finish. We can add an early-return check:

  ```
  if h >= sum(piles):
      return 1
  ```

  This avoids running the binary search entirely for large values of h.

7.7 Pro-Tip: How to Impress the Interviewer

- **Explain the Strict Upper Bound**: Clearly explain why the upper bound `right` is `max(piles)` rather than $\sum$ piles or 10^9. Because Koko cannot eat from two different piles in the same hour, eating at any speed greater than `max(piles)` does not save any hours—it still takes exactly 1 hour per pile. This constraint bounds the search space naturally to max(piles).
- **Avoid CPU Division Delays**: Explain that division `/` and modulo `%` operations are the most expensive arithmetic instructions on modern CPU architectures (often taking 10–40 clock cycles compared to 1 cycle for addition/subtraction). Minimizing divisions or using compiler-friendly integer ceiling tricks showcases low-level system efficiency mindset.

8 Letter Combinations of a Phone Number

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect / QA & Test Engineer
- **Source**: LeetCode 17, Glassdoor
- **Flashcards**: Backtracking deck

8.1 Question Description

Given a string containing digits from `2-9` inclusive, return all possible letter combinations that the number could represent. Return the answer in any order.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that `1` and `0` do not map to any letters.

- `2` -> `"abc"`
- `3` -> `"def"`
- `4` -> `"ghi"`
- `5` -> `"jkl"`
- `6` -> `"mno"`
- `7` -> `"pqrs"`
- `8` -> `"tuv"`
- `9` -> `"wxyz"`

8.1.1 Examples

- **Input**: `digits = "23"`
  - **Output**: `["ad","ae","af","bd","be","bf","cd","ce","cf"]`
- **Input**: `digits = ""`
  - **Output**: `[]`
- **Input**: `digits = "2"`
  - **Output**: `["a","b","c"]`

8.2 Detailed Solution (C++)

The backtracking approach builds combinations index-by-index. For each digit in the input string, we look up its mapped characters, place one character into our current working path, and recursively move to the next digit.

To optimize performance in C++: 1. **Pre-allocated String**: Instead of building the path by repeatedly appending and popping characters (which might trigger reallocations), we pre-allocate a string `path` of size `digits.size()` and overwrite characters by index. 2. **Pass by Reference**: Avoid copying strings and vectors by passing them as references to the helper function.

8.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <string>
#include <algorithm>

class Solution {
private:
    const std::vector<std::string> phone_map = {
        "", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"
    };

    void backtrack(const std::string& digits, int index, std::string& path, std::vector<std::string>& result
        if (index == digits.size()) {
            result.push_back(path);
            return;
        }

        // Convert character digit to index (e.g. '2' - '0' = 2)
        const std::string& letters = phone_map[digits[index] - '0'];
        for (char c : letters) {
            path[index] = c;
            backtrack(digits, index + 1, path, result);
        }
    }
```

```
public:
    std::vector<std::string> letterCombinations(const std::string& digits) {
        if (digits.empty()) {
            return {};
        }

        std::vector<std::string> result;
        std::string path(digits.size(), '\0');

        backtrack(digits, 0, path, result);
        return result;
    }
};
```

8.3 Detailed Solution (Python)

In Python, we implement backtracking using a helper function and a list representation for the mutable path. When we reach a leaf node in the recursion tree (where `index == len(digits)`), we join the list of characters to construct a string and add it to our output.

8.3.1 Standard Python Production Code

```
from typing import List

class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        """
        Generates all possible letter combinations that the number digits could represent.

        Time Complexity: O(4^N * N)
        Space Complexity: O(N)
        """
        if not digits:
            return []

        phone_map = {
            "2": "abc",
            "3": "def",
            "4": "ghi",
            "5": "jkl",
            "6": "mno",
            "7": "pqrs",
```

```
        "8": "tuv",
        "9": "wxyz",
    }

    result: List[str] = []

    def backtrack(index: int, path: List[str]):
        if index == len(digits):
            result.append("".join(path))
            return

        for letter in phone_map[digits[index]]:
            path.append(letter)
            backtrack(index + 1, path)
            path.pop()  # Revert choice for backtracking step

    backtrack(0, [])
    return result
```

8.4 Python-Specific Complexities & Implementation Considerations

8.4.1 1. `join` Overhead

- Using `"".join(path)` takes $\mathcal{O}(N)$ time where N is the length of the list. Since this operation is done at the leaves of the recursion tree, the overall time matches the total output characters generated.
- Avoid using string concatenation (e.g. `path + letter`) in backtracking parameters, as strings are immutable in Python and constructing new strings at each level incurs $\mathcal{O}(N)$ copy cost on every edge of the recursion tree.

8.4.2 2. Space Cost of dynamic lists vs pre-allocation

- Python lists are dynamically resized arrays. Calling `append()` and `pop()` is highly optimized but still has some minor dynamic check overhead.
- For longer strings, pre-allocating a list of fixed size `[None] * len(digits)` and assigning indices directly `path[index] = letter` avoids resize overhead and is slightly faster.

8.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(4^N \cdot N)$	Where N is the number of digits. At most, a digit maps to 4 letters (e.g. '7', '9'). The number of leaves is at most 4^N, and joining/sorting strings takes $\mathcal{O}(N)$ per result.
Space Complexity	$\mathcal{O}(N)$	The depth of the recursion tree is N levels, requiring $\mathcal{O}(N)$ stack space.

8.6 Common Follow-Up Questions & How to Answer

8.6.1 Q1: How can we implement this iteratively instead of using recursion?

- **The Answer**: We can generate combinations iteratively by building them layer by layer. Starting with a list containing an empty string, we iterate over each digit and build new combinations using a list comprehension, which avoids the overhead of `pop(0)` in a standard queue.
- **Python Code**:

```python
def letterCombinationsIterative(digits: str) -> List[str]:
    if not digits:
        return []
    phone_map = {
        "2": "abc", "3": "def", "4": "ghi", "5": "jkl",
        "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz"
    }
    res = [""]
    for digit in digits:
        res = [prev + letter for prev in res for letter in phone_map[digit]]
    return res
```

8.6.2 Q2: What if we want to return a generator/iterator instead of generating all combinations in memory at once?

- **The Answer**: If the digit string is very long, materializing all combinations in memory consumes a massive amount of memory. Using Python's `yield` keywords allows returning a generator that yields combinations lazily.
- **Python Code**:

```python
def letterCombinationsLazy(self, digits: str):
    if not digits:
        return
```

```
phone_map = {"2": "abc", "3": "def", "4": "ghi", "5": "jkl", "6": "mno", "7": "pqrs", "8": "tuv",

def backtrack(index, path):
    if index == len(digits):
        yield "".join(path)
        return
    for letter in phone_map[digits[index]]:
        path.append(letter)
        yield from backtrack(index + 1, path)
        path.pop()
yield from backtrack(0, [])
```

8.7 Pro-Tip: How to Impress the Interviewer

- **Mention Pre-allocation Benefits**: Point out that pre-allocating the string buffer (in C++) or a list (in Python) and mutating elements in-place is highly cache-friendly. It completely eliminates memory allocation and deallocation operations along the recursion path, which is critical for low-latency systems.
- **Discuss Generator Pattern**: Bringing up generators and lazy evaluation shows that you care about memory consumption when handling large-scale datasets, which is highly appreciated in production software design.
- **Address Cache Localization**: Explain that looking up values in a static dictionary/hash map or array is fast, but storing the phone mapping as a contiguous static array or `std::array` instead of a heap-allocated `std::map` avoids pointer indirection and cache misses.

9 Combination Sum

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect / QA & Test Engineer
- **Source**: LeetCode 39, Glassdoor
- **Flashcards**: Backtracking deck

9.1 Question Description

Given an array of **distinct** integers `candidates` and a target integer `target`, return a list of all **unique combinations** of `candidates` where the chosen numbers sum to `target`. You may return the combinations in any order.

The **same** number may be chosen from `candidates` an **unlimited number of times**. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

It is guaranteed that the number of unique combinations that sum up to `target` is less than `150` combinations for the given input.

9.1.1 Examples

- **Input**: `candidates = [2,3,6,7]`, `target = 7`
  - **Output**: `[[2,2,3],[7]]`
  - **Explanation**:
    - `2` and `3` are candidates, and `2 + 2 + 3 = 7`. Note that `2` can be used multiple times.
    - `7` is a candidate, and `7 = 7`.
- **Input**: `candidates = [2,3,5]`, `target = 8`
  - **Output**: `[[2,2,2,2],[2,3,3],[3,5]]`
- **Input**: `candidates = [2]`, `target = 1`
  - **Output**: `[]`

9.2 Detailed Solution (C++)

The backtracking state is defined by the current index `start` in candidates and the `remaining` target sum. Sorting the candidates beforehand is a vital optimization: 1. **Early Pruning**: If `candidates[i]` is greater than `remaining`, then all candidates to its right will also be greater than `remaining` (since the array is sorted). We can immediately `break` the loop, pruning the entire subtree. 2. **Avoiding Duplicates**: We start iterating the loop from the current candidate's index `start`. Since we pass `i` (and not `start` or `0`) as the new `start` in the recursive call, we naturally avoid permutations like `[2, 3, 2]` vs `[2, 2, 3]` and ensure unique combinations.

9.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>

class Solution {
private:
    void backtrack(const std::vector<int>& candidates, int start, int remaining, std::vector<int>& path, std
        if (remaining == 0) {
            result.push_back(path);
            return;
        }

        for (int i = start; i < candidates.size(); ++i) {
            // Early Pruning: candidates is sorted, so subsequent candidates will also exceed remaining
            if (candidates[i] > remaining) {
                break;
            }
```

```
            path.push_back(candidates[i]);
            // i is passed instead of i + 1 because we can reuse the same element
            backtrack(candidates, i, remaining - candidates[i], path, result);
            path.pop_back(); // Backtrack
        }
    }

public:
    std::vector<std::vector<int>> combinationSum(std::vector<int>& candidates, int target) {
        // Sort candidates to enable early pruning
        std::sort(candidates.begin(), candidates.end());

        std::vector<std::vector<int>> result;
        std::vector<int> path;

        backtrack(candidates, 0, target, path, result);
        return result;
    }
};
```

9.3 Detailed Solution (Python)

The Python implementation mirrors this backtracking strategy. We use list slicing `current[:]` or `list(current)` to push a copy of the path to the results list when `remaining == 0`.

9.3.1 Standard Python Production Code

```python
from typing import List

class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:
        """
        Finds all unique combinations in candidates that sum to target.
        Candidates can be used an unlimited number of times.

        Time Complexity: O(N^(T/M + 1))
        Space Complexity: O(T/M)
        """
        result: List[List[int]] = []
        candidates.sort()

        def backtrack(start: int, remaining: int, current: List[int]):
            if remaining == 0:
```

```
            result.append(list(current))
            return

        for i in range(start, len(candidates)):
            # Early Pruning: since candidates is sorted, candidates[i] > remaining
            # means all subsequent elements are also too large.
            if candidates[i] > remaining:
                break

            current.append(candidates[i])
            # Pass i as start index to allow reuse of the current candidate
            backtrack(i, remaining - candidates[i], current)
            current.pop()

    backtrack(0, target, [])
    return result
```

9.4 Python-Specific Complexities & Implementation Considerations

9.4.1 1. `list(current)` vs `current[:]`

- In Python, appending `current` directly to `result` (`result.append(current)`) stores a reference to the mutable list object. As the backtracking algorithm modifies `current`, all entries in `result` would eventually point to the same empty list.
- To prevent this, we must copy the list. Both `result.append(list(current))` and `result.append(current[:])` create a shallow copy. Performance-wise, they are extremely close, but `list()` is often considered more readable and pythonic.

9.4.2 2. Recursion Limit

- The maximum recursion depth occurs when target is formed by using the smallest candidate repeatedly. If `target = 40` and candidate is `2`, the recursion depth is `20`. This is well below Python's default stack depth limit of `1000`, so no `RecursionError` will occur.

9.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(N^{\frac{T}{M}+1})$	Where N is the number of candidates, T is the target, and M is the minimum value in `candidates`. The height of the recursion tree is at most T/M, with a branching factor of at most N. Sorting takes $\mathcal{O}(N \log N)$, which is dominated by backtracking.
Space Complexity	$\mathcal{O}(\frac{T}{M})$	The recursion stack and the path storage `current` can reach a maximum depth of T/M when selecting the smallest candidate repeatedly.

9.6 Common Follow-Up Questions & How to Answer

9.6.1 Q1: What if candidates can only be used at most once, and they might contain duplicate numbers? (Combination Sum II)

- **The Answer**:
  1. Sort the array first.
  2. In the backtracking loop, if the current element is equal to the previous element at the same recursion level, skip it to prevent duplicate combinations: `if (i > start && candidates[i] == candidates[i-1]) continue;`.
  3. Pass `i + 1` to the next recursion level to ensure each number is used at most once.
- **C++ Implementation Snippet**:

```cpp
for (int i = start; i < candidates.size(); ++i) {
    if (candidates[i] > remaining) break;
    if (i > start && candidates[i] == candidates[i-1]) continue; // Skip duplicates
    path.push_back(candidates[i]);
    backtrack(candidates, i + 1, remaining - candidates[i], path, result); // i + 1
    path.pop_back();
}
```

9.6.2 Q2: What if we want to find the number of combinations instead of generating all paths? (Combination Sum IV)

- **The Answer**: Backtracking would be too slow ($\mathcal{O}(N^T)$) and trigger TLE. We can solve it using Dynamic Programming (similar to Unbounded Knapsack/Coin Change). Let `dp[i]` represent the

number of combinations that sum up to `i`. The transition is `dp[i] = sum(dp[i - c])` for each candidate `c`.

- **Python DP Code**:

```python
def combinationSum4(nums: List[int], target: int) -> int:
    dp = [0] * (target + 1)
    dp[0] = 1
    for i in range(1, target + 1):
        for num in nums:
            if i >= num:
                dp[i] += dp[i - num]
    return dp[target]
```

9.7 Pro-Tip: How to Impress the Interviewer

- **Explicitly Highlight Sorting and Early Pruning**: Interviewers look for candidate solutions that optimize early. Explain clearly that sorting candidates reduces the search space significantly by changing the inner loop's condition from an `if` block with a `continue` to a `break`.
- **Discuss Space Optimization**: Show that storing the result elements consumes memory, and in systems with tight memory limits, yielding solutions lazily via C++ generator structures (like standard coroutines in C++20) or Python generator expressions is ideal.

10 Subsets

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 78, Glassdoor
- **Flashcards**: Backtracking deck

10.1 Question Description

Given an integer array `nums` of **unique** elements, return all possible subsets (the power set).

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

10.1.1 Examples

- **Input**: `nums = [1,2,3]`
  - **Output**: `[[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]`
- **Input**: `nums = [0]`
  - **Output**: `[[],[0]]`

10.2 Detailed Solution (C++)

There are two primary paradigms to generate all subsets: **Backtracking** (DFS through the state-space tree) and **Bit Manipulation** (mapping subsets to binary integers).

In C++, backtracking is the most intuitive approach: 1. Start with an empty path. 2. At each recursive call, the current state represents a valid subset, so we add `path` to the `result` immediately. 3. Iterate from the `start` index to the end of the array, push the element, recurse on `i + 1`, and pop to backtrack.

10.2.1 Standard C++ Production Code

```cpp
#include <vector>

class Solution {
private:
    void backtrack(const std::vector<int>& nums, int start, std::vector<int>& path, std::vector<std::vector<
        // Every node visited in the decision tree represents a valid subset
        result.push_back(path);

        for (int i = start; i < nums.size(); ++i) {
            path.push_back(nums[i]);
            backtrack(nums, i + 1, path, result);
            path.pop_back(); // Backtrack
        }
    }

public:
    std::vector<std::vector<int>> subsets(const std::vector<int>& nums) {
        std::vector<std::vector<int>> result;
        std::vector<int> path;
        // Pre-reserve output memory for 2^N elements to avoid vector reallocation
        result.reserve(1 << nums.size());

        backtrack(nums, 0, path, result);
        return result;
    }
};
```

10.3 Detailed Solution (Python)

10.3.1 Standard Python Production Code

```python
from typing import List

class Solution:
```

```python
def subsets(self, nums: List[int]) -> List[List[int]]:
    """
    Generates the power set of a list of unique numbers.

    Time Complexity: O(2^N * N)
    Space Complexity: O(N)
    """
    result: List[List[int]] = []

    def backtrack(start: int, current: List[int]):
        # Append a copy of the current subset
        result.append(list(current))

        for i in range(start, len(nums)):
            current.append(nums[i])
            backtrack(i + 1, current)
            current.pop()  # Backtrack

    backtrack(0, [])
    return result
```

10.4 Python-Specific Complexities & Implementation Considerations

10.4.1 1. Cascading (Iterative) Approach

- Instead of backtracking, Python is extremely well-suited for an iterative, cascading approach. Start with a list containing the empty subset `[[]]`. For each number in `nums`, we iterate through the subsets we have generated so far and create new subsets by adding the number to them.
- **Python Code**:

```python
def subsets(nums: List[int]) -> List[List[int]]:
    result = [[]]
    for num in nums:
        result += [curr + [num] for curr in result]
    return result
```

- This approach is very clean, avoids recursive function call overhead entirely, and does not require explicit backtracking logic.

10.4.2 2. Deep Copies vs. Reference Copying

- Be careful not to append `current` directly in Python. Since `current` is mutated in place, appending `current` will result in a list of empty lists. Use `list(current)` or `current[:]` to create a copy.

10.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(2^n \cdot n)$	There are 2^n total subsets. For each subset, copying the subset of size at most n takes $\mathcal{O}(n)$ time.
Space Complexity	$\mathcal{O}(n)$	The maximum depth of the recursion tree is n. The path vector uses at most $\mathcal{O}(n)$ auxiliary space. (Excluding the output size of $\mathcal{O}(2^n \cdot n)$).

10.6 Common Follow-Up Questions & How to Answer

10.6.1 Q1: How can we implement this using bit manipulation?

- **The Answer**: A subset of a set with n elements can be represented by a bitmask of length n. For each number from 0 to $2^n - 1$, the i-th bit indicates whether the i-th element of the array is present in the subset.
- **C++ Bit Manipulation Implementation**:

```cpp
std::vector<std::vector<int>> subsetsBitmask(const std::vector<int>& nums) {
    int n = nums.size();
    int num_subsets = 1 << n; // 2^n
    std::vector<std::vector<int>> result(num_subsets);

    for (int i = 0; i < num_subsets; ++i) {
        for (int j = 0; j < n; ++j) {
            if ((i >> j) & 1) {
                result[i].push_back(nums[j]);
            }
        }
    }
    return result;
}
```

10.6.2 Q2: What if the elements in `nums` are not unique? (LeetCode 90: Subsets II)

- **The Answer**: We must sort the input array. During backtracking, we skip elements that are identical to the previous element in the loop if they are at the same recursion depth:

```cpp
if (i > start && nums[i] == nums[i - 1]) continue;
```

- This ensures that we only branch on the first occurrence of a duplicate element at each level of the recursion tree.

10.7 Pro-Tip: How to Impress the Interviewer

- **Pre-reserve Vector Capacity**: In C++, standard vectors dynamically double their capacity when they exceed their current limits, which involves allocating new memory blocks and copying elements over. Because we know that the power set size is exactly 2^N, calling `result.reserve(1 << nums.size())` prevents these costly reallocations.
- **Mention Cache Friendliness of Bitwise Manipulation**: The bit manipulation solution is iteratively constructed and can be parallelized since each subset is calculated independently. It also avoids recursion and function call overhead, making it highly competitive on bare-metal architectures.
- **Discuss Space Limits**: If the input size is larger (e.g. $N = 30$), the total number of subsets is $2^{30} \approx 10^9$. Point out that storing the output in memory would crash the program due to Out-Of-Memory (OOM) errors, and in real applications, we should process subsets one-by-one or stream them using generator objects.

11 Matchsticks to Square

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect / QA & Test Engineer
- **Source**: LeetCode 473, Glassdoor
- **Flashcards**: Backtracking deck

11.1 Question Description

You are given an integer array `matchsticks` where `matchsticks[i]` is the length of the i-th matchstick. You want to use **all** the matchsticks to make one square. You should not break any stick, but you can link them up, and each matchstick must be used **exactly one time**.

Return `true` if you can make this square and `false` otherwise.

11.1.1 Examples

- **Input**: `matchsticks = [1,1,2,2,2]`
  - **Output**: `true`
  - **Explanation**: You can form a square of length 2: one side is formed by the two sticks of length 1, and the other three sides are formed by the three sticks of length 2.
- **Input**: `matchsticks = [3,3,3,3,4]`
  - **Output**: `false`
  - **Explanation**: You cannot find a way to form a square using all the matchsticks.

11.1.2 Constraints

- $1 \leq$ matchsticks.length ≤ 15

- $1 \leq \text{matchsticks}[i] \leq 10^8$

11.2 Detailed Solution (C++)

The problem is equivalent to partitioning an array into 4 subsets of equal sum. This is a variation of the Bin Packing / Partition problem, which is NP-complete. We can solve it using backtracking.

To pass within strict execution time limits, we must apply key pruning strategies: 1. **Sum Divisibility Check**: If the sum of all matchsticks is not divisible by 4, it is impossible to form a square. Return `false` immediately. 2. **Descending Sort**: Sort the matchsticks in descending order. Placing larger matchsticks first fills up the sides quickly, triggering failure conditions much earlier in the recursion tree. 3. **Symmetry Breaking**: If we fail to place `matchsticks[idx]` on `sides[i]`, and `sides[i]` is currently `0`, we can `break` and stop trying to place it on subsequent empty sides. Since all empty sides are symmetric, if placing it on `sides[i]` fails, it will also fail on any other empty side.

11.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <numeric>
#include <algorithm>

class Solution {
private:
    bool backtrack(const std::vector<int>& matchsticks, int idx, int side, std::vector<int>& sides) {
        if (idx == matchsticks.size()) {
            // Check if all four sides are equal to the target side length
            return sides[0] == side && sides[1] == side &&
                   sides[2] == side && sides[3] == side;
        }

        for (int i = 0; i < 4; ++i) {
            // Check if placing the matchstick exceeds target side length
            if (sides[i] + matchsticks[idx] <= side) {
                sides[i] += matchsticks[idx];
                if (backtrack(matchsticks, idx + 1, side, sides)) {
                    return true;
                }
                sides[i] -= matchsticks[idx]; // Backtrack

                // Symmetry Breaking: if this side becomes empty after backtracking,
                // and trying it failed, trying subsequent empty sides will also fail.
                if (sides[i] == 0) {
                    break;
                }
```

```
            }
        }
        return false;
    }

public:
    bool makesquare(std::vector<int>& matchsticks) {
        if (matchsticks.empty()) {
            return false;
        }

        long long total = std::accumulate(matchsticks.begin(), matchsticks.end(), 0LL);
        if (total % 4 != 0) {
            return false;
        }

        int side = static_cast<int>(total / 4);

        // Sort descending to optimize search space
        std::sort(matchsticks.begin(), matchsticks.end(), std::greater<int>());
        if (matchsticks[0] > side) {
            return false;
        }

        std::vector<int> sides(4, 0);
        return backtrack(matchsticks, 0, side, sides);
    }
};
```

11.3 Detailed Solution (Python)

11.3.1 Standard Python Production Code

```
from typing import List

class Solution:
    def makesquare(self, matchsticks: List[int]) -> bool:
        """
        Determines if matchsticks can be partitioned into 4 equal sides to form a square.

        Time Complexity: O(4^N) in the worst case, but heavily pruned in practice.
        Space Complexity: O(N)
        """
```

```python
if not matchsticks:
    return False

total = sum(matchsticks)
if total % 4 != 0:
    return False

side = total // 4

# Sort in descending order to prune early
matchsticks.sort(reverse=True)
if matchsticks[0] > side:
    return False

sides = [0, 0, 0, 0]

def backtrack(idx: int) -> bool:
    if idx == len(matchsticks):
        return sides[0] == sides[1] == sides[2] == side

    for i in range(4):
        if sides[i] + matchsticks[idx] <= side:
            sides[i] += matchsticks[idx]
            if backtrack(idx + 1):
                return True
            sides[i] -= matchsticks[idx] # Backtrack

            # Symmetry Breaking: if sides[i] is 0, trying subsequent empty buckets
            # will also fail because empty buckets are identical.
            if sides[i] == 0:
                break
    return False

return backtrack(0)
```

11.4 Python-Specific Complexities & Implementation Considerations

11.4.1 1. Recursion Stack Limit

- The maximum size of `matchsticks` is 15. The recursion depth is bounded by $N = 15$, which is negligible and will not trigger a stack overflow in Python.

11.4.2 2. Python Loop and Sort Performance

- Sorting descending (`matchsticks.sort(reverse=True)`) is exceptionally fast in Python because the Timsort algorithm is implemented in C.
- Python function lookup overhead inside recursion can be slightly optimized by using a nested function `backtrack(idx)`, which allows direct access to the `sides` list and `side` variable in the outer scope, avoiding the overhead of passing them as arguments.

11.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(4^N)$	Where N is the number of matchsticks. Each matchstick has 4 choices. In practice, the search space is heavily reduced by descending sort and symmetry breaking.
Space Complexity	$\mathcal{O}(N)$	The height of the recursion tree is N, requiring $\mathcal{O}(N)$ space on the stack.

11.6 Common Follow-Up Questions & How to Answer

11.6.1 Q1: Can we solve this using Dynamic Programming with Bitmask?

- **The Answer**: Yes, since $N \leq 15$, we can represent the subset of chosen matchsticks using a bitmask. Let `dp[mask]` represent the current side's accumulated length if we use the subset of matchsticks represented by the set bits in `mask`.
- **Bitmask DP (Python) Implementation**:

```python
def makesquareDP(matchsticks: List[int]) -> bool:
    n = len(matchsticks)
    total = sum(matchsticks)
    if total % 4 != 0: return False
    side = total // 4

    # dp[mask] stores the current side's length. -1 means invalid/unreachable state
    dp = [-1] * (1 << n)
    dp[0] = 0

    for mask in range(1 << n):
        if dp[mask] == -1:
```

```
                continue
            for i in range(n):
                # If the i-th matchstick is not used yet
                if not (mask & (1 << i)):
                    next_mask = mask | (1 << i)
                    # If it fits in the current side
                    if dp[mask] + matchsticks[i] <= side:
                        # If we complete one side, reset next state's current sum to 0
                        dp[next_mask] = (dp[mask] + matchsticks[i]) % side

        return dp[(1 << n) - 1] == 0
```

- **Complexity**: Time complexity becomes $\mathcal{O}(N \cdot 2^N)$ and Space Complexity becomes $\mathcal{O}(2^N)$. This is faster for worst-case inputs where backtracking doesn't prune well.

11.6.2 Q2: What if we want to partition the matchsticks into K equal sides instead of 4? (Partition to K Equal Sum Subsets)

- **The Answer**: The problem becomes LeetCode 698. The backtracking algorithm can be generalized by replacing the hardcoded `4` sides with `K` sides (`std::vector<int> sides(K, 0)`). The symmetry breaking condition `if (sides[i] == 0) break;` becomes even more critical because the number of symmetric empty buckets increases.

11.7 Pro-Tip: How to Impress the Interviewer

- **Explain the Mathematics of Symmetry Breaking**: Clearly explain that `sides[i] == 0` is a classic symmetry-breaking optimization. Explain that if we fail to place a matchstick into an empty bucket, placing it into another empty bucket is guaranteed to fail because the state of the buckets is indistinguishable.
- **Discuss Backtracking vs. Bitmask DP**: Demonstrate a deep understanding of trade-offs. Backtracking works best when there is a solution or when early pruning is highly effective (saving memory and average-case CPU cycles). Bitmask DP is more robust against worst-case adversarial inputs because it prevents recalculating overlapping subproblems at the cost of $\mathcal{O}(2^N)$ memory.

12 Zuma Game

- **Category**: Coding
- **Difficulty**: Hard
- **Target Role**: Software Engineer / AI Systems Architect / QA & Test Engineer
- **Source**: LeetCode 488, Glassdoor
- **Flashcards**: Backtracking deck

12.1 Question Description

You are playing a variation of the game Zuma.

There is a single row of colored balls on a board, where each ball can be colored red `'R'`, yellow `'Y'`, blue `'B'`, green `'G'`, or white `'W'`. You also have several colored balls in your hand.

Your goal is to clear all of the balls from the board. On each turn: 1. Choose a ball from your hand and insert it into the row (including the left-most and right-most ends). 2. If the insertion results in a group of **three or more** consecutive balls of the same color, those balls are removed. 3. If this removal causes other groups of three or more consecutive balls of the same color to form, they are also removed. This process (collapsing) continues until no more groups can be removed. 4. Repeat the above steps until either the board is cleared or you run out of balls.

Given a string `board` and a string `hand`, return the **minimum number of balls** you have to insert to clear all the balls from the board. If you cannot clear all the balls, return `-1`.

12.1.1 Examples

- **Input**: `board = "WRRBBW"`, `hand = "RB"`
  - **Output**: `-1`
  - **Explanation**: It is impossible to clear all the balls. The best you can do is:
    - Insert `'R'` so the board becomes `"WRRRBBW"`. `"WRRRBBW"` -> `"WBBW"`.
    - Insert `'B'` so the board becomes `"WBBBW"`. `"WBBBW"` -> `"WW"`.
    - There are still balls remaining on the board, and you are out of balls.
- **Input**: `board = "WWRRBBWW"`, `hand = "WRBRW"`
  - **Output**: `2`
  - **Explanation**: To make the board empty:
    - Insert `'R'` so the board becomes `"WWRRRBBWW"`. `"WWRRRBBWW"` -> `"WWBBWW"`.
    - Insert `'B'` so the board becomes `"WWBBBWW"`. `"WWBBBWW"` -> `"WWWW"` -> `""` (empty).
    - `2` balls from your hand were needed to clear the board.
- **Input**: `board = "G"`, `hand = "GGGGG"`
  - **Output**: `2`

12.1.2 Constraints

- $1 \leq$ board.length ≤ 16
- $1 \leq$ hand.length ≤ 5
- `board` and `hand` consist of the characters `'R'`, `'Y'`, `'B'`, `'G'`, and `'W'`.
- The initial row of balls on the board will not have any groups of three or more consecutive balls of the same color.

12.2 Detailed Solution (C++)

This problem can be modeled as a shortest path search in a state space where each state is a pair of `(board, hand)`. Since we want to find the minimum steps, we can use Backtracking (DFS) with Memoization.

12.2.1 Core Components

1. **Collapse Function**: A helper function `collapse()` recursively removes consecutive groups of 3 or more identical balls from the board.
2. **Memoization Key**: Since the order of balls in our hand does not affect the decisions, we sort the `hand` string. The state can then be represented as a pair: `std::pair<std::string, std::string>` representing `(board, sorted_hand)`.
3. **Backtracking Transitions**: At each step:
   - Collapse the current board.
   - If the board is empty, return 0.
   - If hand is empty (but board is not), return -1.
   - Try inserting every unique ball in our hand at every possible insertion index from 0 to `board.length()`.
   - Recurse on the new board and hand, returning the minimum steps.

12.2.2 Standard C++ Production Code

```cpp
#include <string>
#include <vector>
#include <algorithm>
#include <map>

class Solution {
private:
    // Repeatedly removes any contiguous blocks of 3 or more identical characters
    std::string collapse(const std::string& s) {
        std::string result = s;
        bool changed = true;
        while (changed) {
            changed = false;
            std::string next;
            int i = 0;
            int n = static_cast<int>(result.size());
            while (i < n) {
                int j = i;
                while (j < n && result[j] == result[i]) {
                    j++;
                }
                if (j - i >= 3) {
                    changed = true; // Block of >=3 collapsed
                } else {
                    next.append(result, i, j - i);
                }
                i = j;
            }
```

```
        result = next;
    }
    return result;
}

int dfs(const std::string& board, const std::string& hand,
        std::map<std::pair<std::string, std::string>, int>& memo) {
    std::string b = collapse(board);
    if (b.empty()) return 0;
    if (hand.empty()) return -1;

    auto key = std::make_pair(b, hand);
    if (memo.count(key)) {
        return memo[key];
    }

    int minBalls = -1;
    int blen = static_cast<int>(b.size());
    int hlen = static_cast<int>(hand.size());

    // Try inserting each ball from hand at every possible index
    for (int i = 0; i <= blen; ++i) {
        for (int hi = 0; hi < hlen; ++hi) {
            // Deduplicate: Skip duplicate balls in the sorted hand
            if (hi > 0 && hand[hi] == hand[hi - 1]) {
                continue;
            }

            char color = hand[hi];
            std::string newBoard = b.substr(0, i) + color + b.substr(i);
            std::string newHand = hand.substr(0, hi) + hand.substr(hi + 1);

            int result = dfs(newBoard, newHand, memo);
            if (result != -1) {
                if (minBalls == -1 || result + 1 < minBalls) {
                    minBalls = result + 1;
                }
            }
        }
    }

    memo[key] = minBalls;
    return minBalls;
}
```

```
public:
    int findMinStep(std::string board, std::string hand) {
        // Sorting the hand ensures unique representation of remaining balls
        std::sort(hand.begin(), hand.end());
        std::map<std::pair<std::string, std::string>, int> memo;
        return dfs(board, hand, memo);
    }
};
```

12.3 Detailed Solution (Python)

12.3.1 Standard Python Production Code

```
from typing import Dict, Tuple

class Solution:
    def findMinStep(self, board: str, hand: str) -> int:
        """
        Finds the minimum number of balls to insert to clear the board.

        Time Complexity: O(N * H * H! * (B + H)) in the worst case.
        Space Complexity: O(H! * B) for memoization and call stack.
        """
        def collapse(s: str) -> str:
            changed = True
            while changed:
                changed = False
                i = 0
                while i < len(s):
                    j = i
                    while j < len(s) and s[j] == s[i]:
                        j += 1
                    if j - i >= 3:
                        s = s[:i] + s[j:]
                        changed = True
                    else:
                        i = j
            return s

        memo: Dict[Tuple[str, Tuple[str, ...]], int] = {}

        def dfs(board_str: str, hand_sorted: Tuple[str, ...]) -> int:
```

```
        board_str = collapse(board_str)
        if not board_str:
            return 0
        if not hand_sorted:
            return -1

        key = (board_str, hand_sorted)
        if key in memo:
            return memo[key]

        min_balls = -1

        # Iterate over all possible insertion indices
        for i in range(len(board_str) + 1):
            # Try inserting each ball from our hand
            for hi in range(len(hand_sorted)):
                # Deduplicate: Skip duplicate balls in the sorted hand
                if hi > 0 and hand_sorted[hi] == hand_sorted[hi - 1]:
                    continue

                color = hand_sorted[hi]
                new_board = board_str[:i] + color + board_str[i:]
                new_hand = hand_sorted[:hi] + hand_sorted[hi + 1:]

                result = dfs(new_board, new_hand)
                if result != -1:
                    if min_balls == -1 or result + 1 < min_balls:
                        min_balls = result + 1

        memo[key] = min_balls
        return min_balls

    return dfs(board, tuple(sorted(hand)))
```

12.4 Python-Specific Complexities & Implementation Considerations

12.4.1 1. Hashable Types in Memoization Key

- In Python, dict keys must be immutable (hashable). Thus, we cannot use a list for `hand` in the memo key. We must convert it to a sorted `tuple` (e.g. `tuple(sorted(hand))`) or string.

12.4.2 2. String Concatenation and Performance

- Python string slicing (`s[:i] + color + s[i:]`) produces a new string. Since Zuma's board is small ($B \leq 16$), the overhead is negligible. However, for larger strings, list-based mutation is generally preferred.

12.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}((B+H)! \cdot B)$	Where B is the initial board length and H is the hand length. In the worst case without memoization, we have $B+1$ insertion positions and H choices. Memoization restricts the state space to at most $\mathcal{O}(2^B \cdot 2^H)$ configurations.
Space Complexity	$\mathcal{O}(H! \cdot B)$	The maximum depth of the call stack is H. The space is primarily consumed by the memoization table mapping unique board and hand pairs to results.

12.6 Common Follow-Up Questions & How to Answer

12.6.1 Q1: What are advanced pruning techniques to speed up the execution?

- **The Answer**:
  1. **Adjacent Color Rule**: Only insert a ball of color C next to another ball of color C on the board. Inserting a ball next to a different color is only useful in rare cases (like splitting a pair to trigger secondary collapses), but restricting it to adjacent matches prunes 90% of useless moves.
  2. **Immediate Double Collapse**: If we insert a ball to complete a group of 3, the collapse happens immediately.

12.6.2 Q2: Why is BFS typically preferred over DFS for Zuma-like puzzles?

- **The Answer**: Because we are looking for the **shortest path** to the goal state (empty board). BFS searches level by level (steps of balls used). As soon as BFS finds a state with an empty board, it is guaranteed to be the minimum step solution, allowing us to return immediately and prune all remaining deep branches. DFS, on the other hand, must search the entire tree to ensure the path is optimal.

12.7 Pro-Tip: How to Impress the Interviewer

- **Symmetry and Sorting**: Emphasize that sorting the hand simplifies the state key for memoization. Hand configurations `['R', 'B']` and `['B', 'R']` represent the same state, and sorting merges these identical paths, which is a classic dynamic programming optimization.
- **Identify NP-Hard nature**: Mention that Zuma Game is NP-Hard under general conditions (arbitrary hand size and board length). Acknowledging this helps explain why backtracking with heuristic pruning is the standard and necessary approach rather than a greedy or polynomial-time DP.

13 Non-decreasing Subsequences

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect / QA & Test Engineer
- **Source**: LeetCode 491, Glassdoor
- **Flashcards**: Backtracking deck

13.1 Question Description

Given an integer array `nums`, return all the different possible **non-decreasing subsequences** of the given array with at least two elements. You may return the answer in **any order**.

13.1.1 Examples

- **Input**: `nums = [4,6,7,7]`
  - **Output**: `[[4,6],[4,6,7],[4,6,7,7],[4,7],[4,7,7],[6,7],[6,7,7],[7,7]]`
- **Input**: `nums = [4,4,3,2,1]`
  - **Output**: `[[4,4]]`

13.1.2 Constraints

- $1 \leq$ nums.length ≤ 15
- $-100 \leq$ nums$[i] \leq 100$

13.2 Detailed Solution (C++)

Generating subsequences is a classic backtracking problem. However, there are two tricky constraints here: 1. **Non-decreasing Order**: We can only pick `nums[i]` if it is greater than or equal to the last element in our current path. 2. **No Duplicate Subsequences**: Since `nums` may contain duplicate values (e.g. two `7`s in `[4, 6, 7, 7]`), we might generate the same subsequence multiple times.

Because we **cannot** sort the input array (doing so would change the indices and thus violate the subsequence definition), standard duplicate skipping techniques like `nums[i] == nums[i-1]` cannot be used. Instead: * We use a local `std::unordered_set` (or a flat boolean lookup table) inside each recursion level to track which values have already been chosen as the next element at the current depth.

13.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <unordered_set>
#include <algorithm>

class Solution {
private:
    void backtrack(const std::vector<int>& nums, int start, std::vector<int>& path, std::vector<std::vector<
        // Collect valid non-decreasing subsequences of length >= 2
        if (path.size() >= 2) {
            result.push_back(path);
        }

        // Local lookup set to avoid duplicates at the current level of the decision tree
        std::unordered_set<int> used;

        for (int i = start; i < nums.size(); ++i) {
            // Check if this value has already been branched on at this level
            if (used.count(nums[i])) {
                continue;
            }

            // Ensure non-decreasing constraint
            if (path.empty() || nums[i] >= path.back()) {
                used.insert(nums[i]);
                path.push_back(nums[i]);
                backtrack(nums, i + 1, path, result);
                path.pop_back(); // Backtrack
            }
        }
    }

public:
    std::vector<std::vector<int>> findSubsequences(const std::vector<int>& nums) {
        std::vector<std::vector<int>> result;
        std::vector<int> path;
        backtrack(nums, 0, path, result);
        return result;
```

```
    }
};
```

13.3 Detailed Solution (Python)

13.3.1 Standard Python Production Code

```python
from typing import List

class Solution:
    def findSubsequences(self, nums: List[int]) -> List[List[int]]:
        """
        Finds all different non-decreasing subsequences with at least two elements.

        Time Complexity: O(2^N * N)
        Space Complexity: O(N)
        """
        result: List[List[int]] = []

        def backtrack(start: int, path: List[int]):
            if len(path) >= 2:
                result.append(list(path))

            # Local set to prevent duplicates at the current level of recursion
            used = set()

            for i in range(start, len(nums)):
                if nums[i] in used:
                    continue
                if not path or nums[i] >= path[-1]:
                    used.add(nums[i])
                    path.append(nums[i])
                    backtrack(i + 1, path)
                    path.pop()

        backtrack(0, [])
        return result
```

13.4 Python-Specific Complexities & Implementation Considerations

13.4.1 1. Hash Set Allocation Overhead

- Creating a `set()` at each level of the recursion tree incurs some dynamic memory overhead in Python. Since $N \leq 15$, the recursion depth is small enough that this remains highly performant.
- An alternative representation is to use a simple boolean bitmask array if the values are tightly bounded, which is much faster than hashing.

13.4.2 2. Time-Complexity of `list(path)`

- Similar to other backtracking problems, calling `list(path)` creates a copy of the list. Since the maximum length of `path` is N, each copy takes $\mathcal{O}(N)$ time.

13.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(2^N \cdot N)$	Where N is the number of elements. There are 2^N possible subsequences, and copying a path into the result takes $\mathcal{O}(N)$ time in the worst case.
Space Complexity	$\mathcal{O}(N)$	The height of the recursion stack is at most N. If we count the size of the local deduplication sets, the auxiliary space is bounded by $\mathcal{O}(N^2)$ (since we have at most N active recursion frames, each holding a set of size at most N).

13.6 Common Follow-Up Questions & How to Answer

13.6.1 Q1: Why can't we sort the array first to skip duplicates using `nums[i] == nums[i-1]`?

- **The Answer**: Because the question asks for **subsequences**, which preserves the relative order of elements from the original array. Sorting the array would alter the relative ordering. For example, in `[4, 4, 3, 2, 1]`, the subsequence `[4, 4]` is valid, but if we sorted it to `[1, 2, 3, 4, 4]`, we would generate invalid subsequences like `[1, 2]`.

13.6.2 Q2: How can we implement this without using hash sets at all?

- **The Answer**: Since the values are constrained to $[-100, 100]$, we can use a local fixed-size array/buffer or a bitset instead of an `unordered_set`.

- **C++ Array-Based Optimization**:

```cpp
// Since nums[i] is between -100 and 100, we offset by 100 to map it to [0, 200]
bool used[201] = {false};
for (int i = start; i < nums.size(); ++i) {
    int val_offset = nums[i] + 100;
    if (used[val_offset]) continue;
    if (path.empty() || nums[i] >= path.back()) {
        used[val_offset] = true;
        path.push_back(nums[i]);
        backtrack(nums, i + 1, path, result);
        path.pop_back();
    }
}
```

- This approach avoids heap allocation completely, resulting in a substantial performance boost.

13.7 Pro-Tip: How to Impress the Interviewer

- **Mention Value-Range Offsetting**: Proactively note that the values are constrained to $[-100, 100]$. Pointing out that you can replace the `unordered_set` with a `bool used[201]` array to prevent heap allocation shows strong low-level hardware awareness (cache locality, stack allocation vs heap allocation).
- **Discuss Bitmask Alternatives**: Mention that for very small ranges, bitmasks are extremely efficient because checking/setting a bit takes single-cycle CPU instructions (`OR`, `AND`, `SHIFT`), which are ideal for high-performance and low-latency code.

14 Permutations

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 46, Glassdoor
- **Flashcards**: Subsets deck

14.1 Question Description

Given an array `nums` of **distinct** integers, return all the possible permutations. You can return the answer in **any order**.

14.1.1 Examples

- **Input**: `nums = [1,2,3]`
  - **Output**: `[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

- **Input**: `nums = [0,1]`
  - **Output**: `[[0,1],[1,0]]`
- **Input**: `nums = [1]`
  - **Output**: `[[1]]`

14.1.2 Constraints

- $1 \leq$ nums.length ≤ 6
- $-10 \leq$ nums$[i] \leq 10$
- All the integers of `nums` are **unique**.

14.2 Detailed Solution (C++)

There are two primary ways to generate permutations: 1. **Backtracking with Swapping (In-Place)**: Swap the current element with elements in the range `[start, nums.size() - 1]`, recurse to the next position, and swap back. This method modifies the array in-place, using constant auxiliary space (excluding the recursion stack). 2. **Iterative Cascading (Level-by-Level Insertion)**: Start with an empty permutation `[[]]`. For each number in `nums`, insert it at all possible positions of the previously generated permutations.

We implement the swap-based backtracking approach as it is the most standard, space-efficient, production-grade technique in C++.

14.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <utility>

class Solution {
private:
    void backtrack(std::vector<int>& nums, int start, std::vector<std::vector<int>>& result) {
        // Base case: we have formed a complete permutation
        if (start == nums.size()) {
            result.push_back(nums);
            return;
        }

        for (int i = start; i < nums.size(); ++i) {
            std::swap(nums[start], nums[i]);    // Swapping to place elements at position 'start'
            backtrack(nums, start + 1, result); // Recurse
            std::swap(nums[start], nums[i]);    // Backtrack (restore array)
        }
    }

public:
```

```
    std::vector<std::vector<int>> permute(std::vector<int>& nums) {
        std::vector<std::vector<int>> result;
        backtrack(nums, 0, result);
        return result;
    }
};
```

14.3 Detailed Solution (Python)

In Python, we can write a clean iterative cascading solution that avoids recursion overhead and builds the permutations by inserting the new element into every possible slot of the existing permutations.

14.3.1 Standard Python Production Code

```python
from typing import List

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        """
        Generates all permutations using an iterative cascading approach.

        Time Complexity: O(N! * N)
        Space Complexity: O(N! * N)
        """
        perms = [[]]

        for num in nums:
            new_perms = []
            for p in perms:
                # Insert the current number at every possible index of the existing permutation
                for i in range(len(p) + 1):
                    new_perms.append(p[:i] + [num] + p[i:])
            perms = new_perms

        return perms
```

14.4 Python-Specific Complexities & Implementation Considerations

14.4.1 1. Built-in `itertools.permutations`

- In Python production code, it is almost always better to use `itertools.permutations(nums)`. It is implemented in C and runs substantially faster than hand-written backtracking loops.
- **Python Code**:

```python
import itertools
def permute(nums: List[int]) -> List[List[int]]:
    return [list(p) for p in itertools.permutations(nums)]
```

- In interviews, write the backtracking or cascading approach first, but mention `itertools` to show language proficiency.

14.4.2 2. Space Slicing Penalty

- In the iterative approach, slicing list `p[:i] + [num] + p[i:]` creates three new list objects at each step. While simple, it has higher constant factor overhead compared to in-place backtracking swapping.

14.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n! \cdot n)$	There are $n!$ permutations. For each permutation, copying the array of size n takes $\mathcal{O}(n)$ time.
Space Complexity	$\mathcal{O}(n)$	For swap-based backtracking, the recursion stack depth is n. (Excluding the output size of $\mathcal{O}(n! \cdot n)$).

14.6 Common Follow-Up Questions & How to Answer

14.6.1 Q1: What if duplicates are allowed in the input array? (Permutations II)

- **The Answer**:
  1. Sort the input array `nums`.
  2. Use a boolean array `visited` of size n.
  3. In the backtracking loop, if `nums[i] == nums[i-1]` and `!visited[i-1]`, skip the element to prevent duplicate branches.

- **C++ Code Snippet**:

```cpp
void backtrack(const std::vector<int>& nums, std::vector<bool>& visited, std::vector<int>& path, std::
    if (path.size() == nums.size()) {
        result.push_back(path);
        return;
    }
    for (int i = 0; i < nums.size(); ++i) {
        if (visited[i]) continue;
        // Prevent duplicates
```

```
            if (i > 0 && nums[i] == nums[i - 1] && !visited[i - 1]) continue;

            visited[i] = true;
            path.push_back(nums[i]);
            backtrack(nums, visited, path, result);
            path.pop_back();
            visited[i] = false;
        }
    }
```

14.6.2 Q2: How can we generate permutations in lexicographical order?

- **The Answer**:
  - Swap-based backtracking does not naturally generate permutations in lexicographical order.
  - To get lexicographical order without sorting the final result, we can sort `nums` initially and use `std::next_permutation` in C++, or use the boolean array backtracking strategy where we iterate indices from left to right.
- **C++ next_permutation example**:

```
std::vector<std::vector<int>> permuteLex(std::vector<int>& nums) {
    std::sort(nums.begin(), nums.end());
    std::vector<std::vector<int>> result;
    do {
        result.push_back(nums);
    } while (std::next_permutation(nums.begin(), nums.end()));
    return result;
}
```

14.7 Pro-Tip: How to Impress the Interviewer

- **In-Place Memory Management**: Emphasize that the swap-based method requires no extra memory allocations for local arrays or visited states. This is ideal for system programming or embedded systems where heap allocation is constrained.
- **Mention `std::next_permutation` Internals**: Explain that `std::next_permutation` works by finding the longest non-increasing suffix, locating the pivot to the left of it, swapping the pivot with the smallest element in the suffix that is larger than the pivot, and reversing the suffix. Showing you know the underlying mechanics of standard library functions is highly impressive.

15 Combinations

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect / QA & Test Engineer

- **Source**: LeetCode 77, Glassdoor
- **Flashcards**: Subsets deck

15.1 Question Description

Given two integers `n` and `k`, return all possible combinations of `k` numbers chosen from the range `[1, n]`.

You may return the answer in **any order**.

15.1.1 Examples

- **Input**: `n = 4, k = 2`
  - **Output**: `[[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]`
  - **Explanation**: There are $\binom{4}{2} = 6$ total combinations. Note that combinations are unordered, i.e., `[1,2]` and `[2,1]` are considered to be the same combination.
- **Input**: `n = 1, k = 1`
  - **Output**: `[[1]]`

15.1.2 Constraints

- $1 \leq n \leq 20$
- $1 \leq k \leq n$

15.2 Detailed Solution (C++)

This problem can be modeled as finding all subsets of size `k` from the set $\{1, 2, \ldots, n\}$. A naive backtracking implementation branches on all elements. However, we can perform a highly effective **search space pruning** optimization: * At any state, if the number of elements in our current `path` plus the number of remaining elements in the range `[i, n]` is less than `k`, it is mathematically impossible to construct a combination of size `k`. * Thus, we only loop `i` up to `n - (k - path.size()) + 1`.

15.2.1 Standard C++ Production Code

```cpp
#include <vector>

class Solution {
private:
    void backtrack(int start, int n, int k, std::vector<int>& path, std::vector<std::vector<int>>& result) {
        if (path.size() == k) {
            result.push_back(path);
            return;
        }

        // Optimization: limit the range to avoid building branches that cannot reach size k
```

```
        // Number of elements we still need to select is (k - path.size())
        int limit = n - (k - static_cast<int>(path.size())) + 1;
        for (int i = start; i <= limit; ++i) {
            path.push_back(i);
            backtrack(i + 1, n, k, path, result);
            path.pop_back(); // Backtrack
        }
    }

public:
    std::vector<std::vector<int>> combine(int n, int k) {
        std::vector<std::vector<int>> result;
        std::vector<int> path;
        backtrack(1, n, k, path, result);
        return result;
    }
};
```

15.3 Detailed Solution (Python)

15.3.1 Standard Python Production Code

```
from typing import List

class Solution:
    def combine(self, n: int, k: int) -> List[List[int]]:
        """
        Generates all combinations of k numbers from [1, n].

        Time Complexity: O(k * C(n, k))
        Space Complexity: O(k)
        """
        result: List[List[int]] = []

        def backtrack(start: int, path: List[int]):
            if len(path) == k:
                result.append(list(path))
                return

            # Optimization: prune branches where remaining elements are insufficient
            limit = n - (k - len(path)) + 1
            for i in range(start, limit + 1):
                path.append(i)
```

```
            backtrack(i + 1, path)
            path.pop()  # Backtrack

    backtrack(1, [])
    return result
```

15.4 Python-Specific Complexities & Implementation Considerations

15.4.1 1. Built-in `itertools.combinations`

- Python's standard library provides `itertools.combinations(range(1, n + 1), k)`. This is implemented in C and is significantly faster than any user-space recursive function.
- **Python Code**:

```
import itertools
def combine(n: int, k: int) -> List[List[int]]:
    return [list(c) for c in itertools.combinations(range(1, n + 1), k)]
```

- Always mention this built-in capability in interviews to show you write production-grade code.

15.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}\left(k \cdot \binom{n}{k}\right)$	There are $\binom{n}{k}$ combinations. For each combination, we copy the path of size k into the result.
Space Complexity	$\mathcal{O}(k)$	The depth of the recursion tree is at most k, requiring $\mathcal{O}(k)$ stack space.

15.6 Common Follow-Up Questions & How to Answer

15.6.1 Q1: How can we implement this iteratively (without recursion)?

- **The Answer**: We can generate combinations in lexicographical order. Start with the first combination `[1, 2, ..., k]`. To find the next, we find the rightmost index `i` that is less than `n - k + i + 1`, increment it, and reset all elements to its right to be consecutive values.
- **C++ Iterative Code**:

```
std::vector<std::vector<int>> combineIterative(int n, int k) {
    std::vector<std::vector<int>> result;
    std::vector<int> path(k, 0);
```

```
    int i = 0;
    while (i >= 0) {
        path[i]++;
        if (path[i] > n - k + i + 1) {
            i--;
        } else if (i == k - 1) {
            result.push_back(path);
        } else {
            i++;
            path[i] = path[i - 1];
        }
    }
    return result;
}
```

15.6.2 Q2: How can we generate combinations using bit manipulation (Gosper's Hack)?

- **The Answer**: Gosper's Hack is an advanced bit manipulation algorithm that finds the next integer with the same number of set bits (k bits).
- **C++ Gosper's Hack Snippet**:

```
std::vector<std::vector<int>> combineGosper(int n, int k) {
    std::vector<std::vector<int>> result;
    int mask = (1 << k) - 1;
    int limit = 1 << n;
    while (mask < limit) {
        std::vector<int> path;
        for (int i = 0; i < n; ++i) {
            if ((mask >> i) & 1) path.push_back(i + 1);
        }
        result.push_back(path);

        // Gosper's Hack transition
        int c = mask & -mask;
        int r = mask + c;
        mask = (((r ^ mask) >> 2) / c) | r;
    }
    return result;
}
```

15.7 Pro-Tip: How to Impress the Interviewer

- **In-Place Array Pruning Boundary**: Proactively explain the math behind the loop upper limit: `n - (k - path.size()) + 1`. This shows you are not just writing standard recursion but are actively

optimizing the loop boundaries to prune dead search spaces.

- **Discuss Gosper's Hack**: Mentioning Gosper's Hack in a systems/low-level developer interview is a major plus. It proves you understand bitwise arithmetic and know how to traverse set bits in $O(1)$ transitions per state.

16 Subsets II

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 90, Glassdoor
- **Flashcards**: Subsets deck

16.1 Question Description

Given an integer array `nums` that may contain duplicate elements, return all possible subsets (the power set).

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

16.1.1 Examples

- **Input**: `nums = [1,2,2]`
  - **Output**: `[[],[1],[1,2],[1,2,2],[2],[2,2]]`
- **Input**: `nums = [0]`
  - **Output**: `[[],[0]]`

16.1.2 Constraints

- $1 \leq$ nums.length ≤ 10
- $-10 \leq$ nums$[i] \leq 10$

16.2 Detailed Solution (C++)

To prevent duplicate subsets without resorting to slow post-generation deduplication (e.g. using `std::set`), we sort the array first. Sorting places duplicate elements contiguously.

During backtracking: 1. Every node in the recursion tree represents a valid subset, so we push the current `path` to the `result` list immediately. 2. In the loop `for (int i = start; i < nums.size(); ++i)`, if `i > start` and `nums[i] == nums[i-1]`, we `continue` (skip). This ensures that we do not initiate duplicate branches at the same recursion depth.

16.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>

class Solution {
private:
    void backtrack(const std::vector<int>& nums, int start, std::vector<int>& path, std::vector<std::vector<
        // Every node visited in the decision tree is a unique subset
        result.push_back(path);

        for (int i = start; i < nums.size(); ++i) {
            // Skip duplicates at the same recursion depth
            if (i > start && nums[i] == nums[i - 1]) {
                continue;
            }
            path.push_back(nums[i]);
            backtrack(nums, i + 1, path, result);
            path.pop_back(); // Backtrack
        }
    }

public:
    std::vector<std::vector<int>> subsetsWithDup(std::vector<int>& nums) {
        // Sort to bring duplicates together
        std::sort(nums.begin(), nums.end());

        std::vector<std::vector<int>> result;
        std::vector<int> path;

        // Since there are duplicates, the actual number of subsets is <= 2^N
        result.reserve(1 << nums.size());

        backtrack(nums, 0, path, result);
        return result;
    }
};
```

16.3 Detailed Solution (Python)

16.3.1 Standard Python Production Code

```python
from typing import List
```

```python
class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        """
        Generates the power set of nums without duplicate subsets.

        Time Complexity: O(2^N * N)
        Space Complexity: O(N)
        """
        result: List[List[int]] = []
        nums.sort()

        def backtrack(start: int, current: List[int]):
            result.append(list(current))

            for i in range(start, len(nums)):
                # If the current element is a duplicate of the previous
                # and we are not at the starting index of this recursion level, skip
                if i > start and nums[i] == nums[i - 1]:
                    continue

                current.append(nums[i])
                backtrack(i + 1, current)
                current.pop()  # Backtrack

        backtrack(0, [])
        return result
```

16.4 Python-Specific Complexities & Implementation Considerations

16.4.1 1. Iterative Cascading Approach with Duplicates

- We can generate subsets iteratively. If we sort `nums`, duplicate elements will be adjacent. When processing a duplicate element `nums[i]`, we only append it to the subsets that were created in the **previous step**. This is a highly efficient way to avoid generating duplicate subsets without recursion.
- **Python Code**:

```python
def subsetsWithDupIterative(nums: List[int]) -> List[List[int]]:
    nums.sort()
    result = [[]]
    prev_size = 0
    for i in range(len(nums)):
        # If current element is duplicate, start appending from the subsets
        # created in the previous step (prev_size)
        start = prev_size if (i > 0 and nums[i] == nums[i-1]) else 0
```

```
            prev_size = len(result)
            for j in range(start, prev_size):
                result.append(result[j] + [nums[i]])
        return result
```

- This approach completely avoids recursion overhead and has a smaller constant factor.

16.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(2^n \cdot n)$	Where n is the number of elements. In the worst case (no duplicates), there are 2^n subsets, and copying each takes $\mathcal{O}(n)$ time. Sorting takes $\mathcal{O}(n \log n)$.
Space Complexity	$\mathcal{O}(n)$	The recursion stack and path array require $\mathcal{O}(n)$ space. (Excluding the output size of $\mathcal{O}(2^n \cdot n)$).

16.6 Common Follow-Up Questions & How to Answer

16.6.1 Q1: Why does `i > start` work to skip duplicates at the same level?

- **The Answer**:
  - `i > start` ensures we only skip duplicates at the **same recursion level** (in the loop).
  - If `i == start`, it is the first time we are picking this value for the current path slot, which is valid and necessary.
  - If `i > start` and `nums[i] == nums[i - 1]`, it means we already branched on this value at the current depth in a previous iteration of the loop. Branching on it again would generate redundant subsets.

16.6.2 Q2: What if we want to return the subsets sorted by size?

- **The Answer**: We can sort the output array of lists using a custom comparator. In Python: `result.sort(key=len)`. In C++: `std::sort(result.begin(), result.end(), [](const std::vector<int>& a, const std::vector<int>& b) { return a.size() < b.size(); });`.

16.7 Pro-Tip: How to Impress the Interviewer

- **Explain the Math behind Iterative Cascading with Duplicates**: Presenting the iterative cascading approach with the `prev_size` optimization shows an exceptionally deep understanding of

combinatorial generation. It is elegant and proves you can write highly optimized, non-recursive code.

- **Explain Memory Reserving**: Emphasize that calling `reserve(1 << nums.size())` in C++ allocates memory for the maximum possible subsets upfront, eliminating the costly overhead of dynamic array resizing and vector copying during backtracking.

17 Implement Trie (Prefix Tree)

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 208, Glassdoor
- **Flashcards**: Trie deck

17.1 Question Description

A **trie** (pronounced as "try") or **prefix tree** is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete, spellcheckers, and IP routing tables.

Implement the `Trie` class:

- `Trie()` Initializes the trie object.
- `void insert(String word)` Inserts the string `word` into the trie.
- `boolean search(String word)` Returns `true` if the string `word` is in the trie (i.e., was inserted before), and `false` otherwise.
- `boolean startsWith(String prefix)` Returns `true` if there is a previously inserted string `word` that has the prefix `prefix`, and `false` otherwise.

17.1.1 Examples

- **Input**:

```
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
```

- **Output**:

```
[null, null, true, false, true, null, true]
```

- **Explanation**:

```
Trie trie;
trie.insert("apple");
trie.search("apple");   // returns true
trie.search("app");     // returns false
trie.startsWith("app"); // returns true
```

```
trie.insert("app");
trie.search("app");     // returns true
```

17.1.2 Constraints

- $1 \leq \text{word.length}, \text{prefix.length} \leq 2000$
- `word` and `prefix` consist only of lowercase English letters.
- At most 3×10^4 calls in total will be made to `insert`, `search`, and `startsWith`.

17.2 Detailed Solution (C++)

For lowercase English letters, using an array of 26 pointers (`TrieNode* children[26]`) is far more efficient than `std::unordered_map`. It avoids hash collision overhead, eliminates map metadata memory, and guarantees $\mathcal{O}(1)$ child lookup using simple pointer arithmetic.

We must also ensure proper memory management in C++ by implementing the **Rule of Five** (handling or deleting copy operations) to prevent double-free vulnerabilities when a `Trie` is copied.

17.2.1 Standard C++ Production Code

```cpp
#include <string>
#include <string_view>
#include <utility>

class Trie {
private:
    struct TrieNode {
        TrieNode* children[26] = {nullptr};
        bool is_end = false;

        // Recursive destructor cleanly deallocates the entire trie tree
        ~TrieNode() {
            for (TrieNode* child : children) {
                delete child;
            }
        }
    };

    TrieNode* root;

    // Helper method to traverse the Trie and locate a prefix
    TrieNode* find(std::string_view prefix) const noexcept {
        TrieNode* cur = root;
        for (char ch : prefix) {
            int idx = ch - 'a';
```

```
            if (!cur->children[idx]) {
                return nullptr;
            }
            cur = cur->children[idx];
        }
        return cur;
    }

public:
    Trie() : root(new TrieNode()) {}

    ~Trie() {
        delete root;
    }

    // Disable copy semantics to prevent shallow-copy double-free issues
    Trie(const Trie&) = delete;
    Trie& operator=(const Trie&) = delete;

    // Enable move semantics for transfer of ownership
    Trie(Trie&& other) noexcept : root(other.root) {
        other.root = nullptr;
    }

    Trie& operator=(Trie&& other) noexcept {
        if (this != &other) {
            delete root;
            root = other.root;
            other.root = nullptr;
        }
        return *this;
    }

    void insert(const std::string& word) {
        TrieNode* cur = root;
        for (char ch : word) {
            int idx = ch - 'a';
            if (!cur->children[idx]) {
                cur->children[idx] = new TrieNode();
            }
            cur = cur->children[idx];
        }
        cur->is_end = true;
    }
```

```
    bool search(const std::string& word) const noexcept {
        const TrieNode* node = find(word);
        return node != nullptr && node->is_end;
    }

    bool startsWith(const std::string& prefix) const noexcept {
        return find(prefix) != nullptr;
    }
};
```

17.3 Detailed Solution (Python)

17.3.1 Standard Python Production Code

Below is the type-hinted Python implementation. It uses a Python `dict` for child lookups. To minimize memory, `__slots__` is declared on the `TrieNode` class.

```
from typing import Optional

class TrieNode:
    # Use __slots__ to eliminate dynamic instance dictionary overhead
    __slots__ = ("children", "is_end")

    def __init__(self) -> None:
        self.children: dict[str, TrieNode] = {}
        self.is_end: bool = False

class Trie:
    def __init__(self) -> None:
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        """
        Inserts a word into the trie.
        Time Complexity: O(L) where L is word length
        Space Complexity: O(L) in the worst case (new nodes created)
        """
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True
```

```python
    def search(self, word: str) -> bool:
        """
        Returns true if the word is in the trie.
        Time Complexity: O(L) where L is word length
        """
        node = self._find(word)
        return node is not None and node.is_end

    def startsWith(self, prefix: str) -> bool:
        """
        Returns true if there is any word in the trie that starts with the given prefix.
        Time Complexity: O(P) where P is prefix length
        """
        return self._find(prefix) is not None

    def _find(self, prefix: str) -> Optional[TrieNode]:
        """
        Helper method to search for a node matching a prefix.
        """
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node
```

17.4 Python-Specific Complexities & Implementation Considerations

17.4.1 1. The Power of `__slots__`

- In Python, standard class instances store their attributes in a dynamic dictionary (`__dict__`). This incurs significant memory overhead (at least 104 bytes per dictionary + pointer overhead).
- A Trie typically contains thousands or millions of nodes. By defining `__slots__ = ("children", "is_end")` on `TrieNode`, we instruct Python to allocate space only for the declared references, decreasing the memory footprint of the trie by **up to 70%**.

17.4.2 2. Hash Map vs Fixed-Size List

- Python's `dict` uses a highly optimized hash map internally. For 26 elements, it is extremely fast and space-efficient because it only stores keys that actually exist.
- In contrast, allocating a list of size 26 (`[None] * 26`) for every single node allocates 26 references upfront, which is memory-wasteful for sparse trees where nodes have low out-degrees.

17.4.3 3. Reusing Prefix Lookup Logic

- Avoid duplicating the traversal loop in `search` and `startsWith`. Writing a helper `_find(prefix)` returns the exact node where the prefix ends, letting `search` simply inspect `.is_end` and `startsWith` check if the returned node is not `None`.

17.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity (insert)	$\mathcal{O}(L)$	L is the length of the word; we traverse or create at most L nodes.
Time Complexity (search)	$\mathcal{O}(L)$	L is the length of the word; we trace the path in the trie.
Time Complexity (startsWith)	$\mathcal{O}(P)$	P is the length of the prefix.
Space Complexity	$\mathcal{O}(N \cdot L)$	N is the number of words, L is the average length. Words with common prefixes share nodes, saving space.

17.6 Common Follow-Up Questions & How to Answer

17.6.1 Q1: How would you make this Trie thread-safe?

- **The Problem**: Multiple threads inserting and searching concurrently can lead to race conditions (e.g., two threads inserting different words sharing the same prefix may overwrite each other's child nodes).
- **The Solution**:
  1. **Coarse-grained Mutex**: Lock the entire Trie for write operations. This severely limits throughput under write-heavy workloads.
  2. **Read-Write Mutex (`std::shared_mutex`)**: Allows concurrent reads (`search`, `startsWith`) but exclusive locks for writes (`insert`).
  3. **Fine-grained Node Locking**: Put a mutex on each individual `TrieNode`. A thread only locks the child pointer array when modifying a specific node.
  4. **Lock-Free Trie**: Implement using atomic pointer checks (`std::atomic<TrieNode*>`). When a thread wants to initialize a null child node, it uses a Compare-And-Swap (`compare_exchange_weak`) operation. If another thread beat it to initialization, it deletes its local allocation and uses the existing node.

17.6.2 Q2: What if we want to store Unicode characters (e.g. UTF-8)?

- **The Problem**: A raw array of size 26 is limited to ASCII lowercase English. Unicode characters have a vast space (over 1.1 million code points).
- **The Solution**: Change the node's representation of children.
  - **Hash Map**: Use `std::unordered_map<char32_t, TrieNode*>` in C++ or standard `dict` in Python.
  - **Ternary Search Tree (TST)**: Instead of multi-way nodes, each node contains a single character and three pointers (left, middle, right), balancing binary search tree lookup with Trie prefix sharing.

17.7 Pro-Tip: How to Impress the Interviewer

- **Highlight RAII and the Rule of Five**: In C++, explain how a custom recursive destructor in the node (`~TrieNode`) automatically handles the deletion of the entire sub-tree when the parent `Trie` goes out of scope. Explicitly deleting copy operations prevents double-frees.
- **Discuss Memory Fragmentation & Cache Locality**: A standard pointer-based Trie is notorious for **cache misses** because each node is allocated dynamically on the heap (`new`), spreading them across arbitrary memory locations. Point out that allocating nodes contiguously inside a single `std::vector<TrieNode>` block (using indices instead of raw pointers) dramatically improves CPU cache hits.

18 Design Add and Search Words Data Structure

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect
- **Source**: LeetCode 211, Glassdoor
- **Flashcards**: Trie deck

18.1 Question Description

Design a data structure that supports adding new words and finding if a string matches any previously added string.

Implement the `WordDictionary` class:

- `WordDictionary()` Initializes the object.
- `void addWord(word)` Adds `word` to the data structure, it can be matched later.
- `bool search(word)` Returns `true` if there is any string in the data structure that matches `word` or `false` otherwise. `word` may contain dots `.` where dots can be matched with any letter.

18.1.1 Examples

- **Input**:

```
["WordDictionary","addWord","addWord","addWord","search","search","search","search"]
[[],["bad"],["dad"],["mad"],["pad"],["bad"],[".ad"],["b.."]]
```

- **Output**:

```
[null,null,null,null,false,true,true,true]
```

- **Explanation**:

```
WordDictionary wordDictionary;
wordDictionary.addWord("bad");
wordDictionary.addWord("dad");
wordDictionary.addWord("mad");
wordDictionary.search("pad"); // return False
wordDictionary.search("bad"); // return True
wordDictionary.search(".ad"); // return True
wordDictionary.search("b.."); // return True
```

18.1.2 Constraints

- $1 \leq$ word.length ≤ 25
- `word` in `addWord` consists of lowercase English letters.
- `word` in `search` consists of `.` or lowercase English letters.
- There will be at most 2 dots in `word` for search queries.
- At most 10^4 calls in total will be made to `addWord` and `search`.

18.2 Detailed Solution (C++)

To implement the wildcard search feature (`.`), we can build a Trie. Standard word insertions are identical to a regular Trie. For searching: 1. If the character is a lowercase English letter, we retrieve the corresponding child pointer `children[ch - 'a']`. 2. If the character is `.`, we perform a **Depth-First Search (DFS)**, recursively trying all non-null children pointers at the current depth.

18.2.1 Standard C++ Production Code

```cpp
#include <string>
#include <string_view>
#include <utility>

class WordDictionary {
private:
    struct TrieNode {
        TrieNode* children[26] = {nullptr};
```

```
        bool is_end = false;

        // Cleanly deallocate child nodes recursively
        ~TrieNode() {
            for (TrieNode* child : children) {
                delete child;
            }
        }
    };

    TrieNode* root;

    // Helper recursive method for wildcard matching
    bool dfs(const TrieNode* node, std::string_view word, size_t index) const noexcept {
        if (index == word.size()) {
            return node->is_end;
        }

        char ch = word[index];
        if (ch == '.') {
            // Wildcard match: explore all possible paths
            for (const TrieNode* child : node->children) {
                if (child && dfs(child, word, index + 1)) {
                    return true;
                }
            }
            return false;
        } else {
            int idx = ch - 'a';
            const TrieNode* child = node->children[idx];
            return child != nullptr && dfs(child, word, index + 1);
        }
    }

public:
    WordDictionary() : root(new TrieNode()) {}

    ~WordDictionary() {
        delete root;
    }

    // Disable copy operations to ensure memory safety
    WordDictionary(const WordDictionary&) = delete;
    WordDictionary& operator=(const WordDictionary&) = delete;
```

```
    // Support move operations
    WordDictionary(WordDictionary&& other) noexcept : root(other.root) {
        other.root = nullptr;
    }

    WordDictionary& operator=(WordDictionary&& other) noexcept {
        if (this != &other) {
            delete root;
            root = other.root;
            other.root = nullptr;
        }
        return *this;
    }

    void addWord(const std::string& word) {
        TrieNode* cur = root;
        for (char ch : word) {
            int idx = ch - 'a';
            if (!cur->children[idx]) {
                cur->children[idx] = new TrieNode();
            }
            cur = cur->children[idx];
        }
        cur->is_end = true;
    }

    bool search(const std::string& word) const noexcept {
        return dfs(root, word, 0);
    }
};
```

18.3 Detailed Solution (Python)

18.3.1 Standard Python Production Code

Below is the type-hinted, recursive Python implementation. It uses `__slots__` to optimize memory and avoids string slicing to maintain efficiency.

```
class TrieNode:
    __slots__ = ("children", "is_end")

    def __init__(self) -> None:
        self.children: dict[str, TrieNode] = {}
```

```
        self.is_end: bool = False

class WordDictionary:
    def __init__(self) -> None:
        self.root = TrieNode()

    def addWord(self, word: str) -> None:
        """
        Adds a word to the trie.
        Time Complexity: O(L)
        """
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def search(self, word: str) -> bool:
        """
        Returns true if there is any string in the trie matching the word.
        '.' can match any lowercase English letter.
        """
        return self._dfs(self.root, word, 0)

    def _dfs(self, node: TrieNode, word: str, index: int) -> bool:
        # Base Case: We reached the end of the query string
        if index == len(word):
            return node.is_end

        ch = word[index]
        if ch == ".":
            # Wildcard branch: Try all available children
            for child in node.children.values():
                if self._dfs(child, word, index + 1):
                    return True
            return False

        # Exact character match
        if ch not in node.children:
            return False
        return self._dfs(node.children[ch], word, index + 1)
```

18.4 Python-Specific Complexities & Implementation Considerations

18.4.1 1. Avoiding the String Slicing Trap

- A common Python mistake during DFS searches is passing a sliced string to subsequent recursion frames: `self._dfs(child, word[1:])`.
- Slicing a string of length L creates a shallow copy, taking $\mathcal{O}(L)$ time and space. Doing this on every level of the recursive tree results in $\mathcal{O}(L^2)$ time/space overhead.
- By passing the original string and a tracking index (`index + 1`), we keep string references constant and guarantee $\mathcal{O}(1)$ space per call frame.

18.4.2 2. Fast Wildcard Iteration with `.values()`

- When evaluating `.`, iterating directly over `node.children.values()` is faster than doing key lookups `node.children[key]` because it avoids hashing keys and traverses the underlying dictionary table values directly in C-speed.

18.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity (addWord)	$\mathcal{O}(L)$	We traverse the characters of the word and insert new nodes.
Time Complexity (search)	$\mathcal{O}(M^L)$	In the worst case (query contains all `.` wildcards), we may visit all nodes in the Trie. M is the branching factor (up to 26). For queries without wildcards, it is $\mathcal{O}(L)$.
Space Complexity	$\mathcal{O}(N \cdot L)$	Dynamic heap memory for the Trie structure. Auxiliary space for the search stack is $\mathcal{O}(L)$ where L is the length of the query.

18.6 Common Follow-Up Questions & How to Answer

18.6.1 Q1: How would you optimize the search if the dictionary contains millions of words, and we get queries with many dots?

- **The Problem**: Searching for `.....` in a dense Trie containing millions of words requires examining almost every node, leading to severe slowdowns.
- **The Solutions**:

1. **Length Pruning**: Keep track of the lengths of all added words. In Python, maintain a set of integer lengths (`self.lengths = set()`). Before executing a search, check if `len(word) in self.lengths`. If not, return `false` instantly.
2. **Separate Trees by Length**: Maintain a hash map of `word_length -> TrieRoot`. We only search the Trie corresponding to the exact length of the search word, pruning unrelated branches.

18.6.2 Q2: What if we want to search words matching a regular expression beyond '.' (e.g. '*' or '[a-z]')?

- **The Solution**: The Trie DFS structure can easily adapt to other patterns:
  - For character classes like `[a-z]`: Extract the characters inside the brackets and recursively search only those children.
  - For kleene star $*$: We can either remain at the current node (match 0 characters) or try matching the character multiple times.

18.7 Pro-Tip: How to Impress the Interviewer

- **Mention length-filtering optimization upfront**: Interviewers love candidates who identify bottleneck cases (like a search query consisting of all dots) and immediately propose cheap checks like `length` filtering to bypass expensive traversals.
- **Rule of Five (C++)**: Explicitly write a copy constructor and assignment operator deletion. This demonstrates your awareness of safe memory practices in C++ to prevent double-frees when local nodes go out of scope.

19 Word Search II

- **Category**: Coding
- **Difficulty**: Hard
- **Target Role**: Software Engineer / AI Systems Architect / QA & Test Engineer
- **Source**: LeetCode 212, Glassdoor
- **Flashcards**: Trie deck

19.1 Question Description

Given an $m \times n$ board of characters and a list of strings `words`, return *all words on the board*.

Each word must be constructed from letters of sequentially adjacent cells, where **adjacent cells** are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

19.1.1 Examples

- **Input**:

```
board = [
  ["o","a","a","n"],
  ["e","t","a","e"],
  ["i","h","k","r"],
  ["i","f","l","v"]
],
words = ["oath","pea","eat","rain"]
```

  - **Output**: `["eat","oath"]`

- **Input**:

```
board = [
  ["a","b"],
  ["c","d"]
],
words = ["abcb"]
```

  - **Output**: `[]`

19.1.2 Constraints

- $m ==$ board.length
- $n ==$ board$[i]$.length
- $1 \leq m, n \leq 12$
- `board[i][j]` is a lowercase English letter.
- $1 \leq$ words.length $\leq 3 \times 10^4$
- $1 \leq$ words$[i]$.length ≤ 10
- `words[i]` consists of lowercase English letters.
- All strings in `words` are unique.

19.2 Detailed Solution (C++)

Running a standard depth-first search for each word individually on the board results in $O(W \cdot M \cdot N \cdot 4^L)$ complexity (where W is the number of words and L is word length), which triggers a Time Limit Exceeded (TLE) error.

Instead, we can search all words simultaneously by loading them into a **Trie** and traversing the board and the Trie together.

19.2.1 Essential Performance Optimizations

1. **Store the Whole Word at the Leaf**: Instead of building the prefix character-by-character as we traverse the board, we store the full string at the terminal node (`node->word = word`). Once matched, we add it to the results and clear it to prevent duplicates.

2. **Backtracking with In-Place Visited Marks**: We temporarily write `#` to `board[r][c]` to indicate it is visited, avoiding the overhead of allocating a separate `visited` set or grid. We restore it before backtracking.
3. **Trie Pruning (The Game Changer)**: Once a word is found, its node is no longer needed. If a leaf node has no remaining children, we can delete it and prune the pointer in its parent node. This prevents future searches from traversing dead-ends on the board.

19.2.2 Standard C++ Production Code

```cpp
#include <vector>
#include <string>
#include <array>
#include <utility>

class Solution {
private:
    struct TrieNode {
        TrieNode* children[26] = {nullptr};
        std::string word = "";

        // Recursive destructor safely cleans up all remaining children on destruction
        ~TrieNode() {
            for (TrieNode* child : children) {
                delete child;
            }
        }
    };

    int rows = 0;
    int cols = 0;
    std::vector<std::string> result;

    void dfs(std::vector<std::vector<char>>& board, int r, int c, TrieNode* parent, int idx) {
        TrieNode* curr = parent->children[idx];
        if (!curr) return;

        // Found a word
        if (!curr->word.empty()) {
            result.push_back(curr->word);
            curr->word.clear(); // Mark as found to avoid duplicate results
        }

        // Cache character and mark cell as visited
        char ch = board[r][c];
```

```
        board[r][c] = '#';

        // 4-Directional DFS
        static constexpr int dr[] = {0, 0, 1, -1};
        static constexpr int dc[] = {1, -1, 0, 0};

        for (int d = 0; d < 4; ++d) {
            int nr = r + dr[d];
            int nc = c + dc[d];
            if (nr >= 0 && nr < rows && nc >= 0 && nc < cols) {
                char next_ch = board[nr][nc];
                if (next_ch != '#') {
                    dfs(board, nr, nc, curr, next_ch - 'a');
                }
            }
        }

        // Backtrack: restore character
        board[r][c] = ch;

        // Trie Pruning: If this node has no children left, prune it from parent
        bool has_children = false;
        for (const TrieNode* child : curr->children) {
            if (child) {
                has_children = true;
                break;
            }
        }

        if (!has_children) {
            delete curr; // Free memory of this branch
            parent->children[idx] = nullptr;
        }
    }

public:
    std::vector<std::string> findWords(std::vector<std::vector<char>>& board, std::vector<std::string>& word
        if (board.empty() || board[0].empty() || words.empty()) {
            return {};
        }

        rows = static_cast<int>(board.size());
        cols = static_cast<int>(board[0].size());
        result.clear();
```

```
        // 1. Build the Trie
        TrieNode root;
        for (const std::string& word : words) {
            TrieNode* cur = &root;
            for (char ch : word) {
                int idx = ch - 'a';
                if (!cur->children[idx]) {
                    cur->children[idx] = new TrieNode();
                }
                cur = cur->children[idx];
            }
            cur->word = word;
        }

        // 2. Search starting from each cell
        for (int r = 0; r < rows; ++r) {
            for (int c = 0; c < cols; ++c) {
                char ch = board[r][c];
                int idx = ch - 'a';
                if (root.children[idx]) {
                    dfs(board, r, c, &root, idx);
                }
            }
        }

        return result;
    }
};
```

19.3 Detailed Solution (Python)

19.3.1 Standard Python Production Code

Below is the optimized Python version. It leverages dictionary key checks for fast lookups and prunes child nodes dynamically using `.pop()`.

```
from typing import List, Optional

class TrieNode:
    __slots__ = ("children", "word")

    def __init__(self) -> None:
        self.children: dict[str, TrieNode] = {}
```

```
        self.word: Optional[str] = None

class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]:
        if not board or not board[0] or not words:
            return []

        rows, cols = len(board), len(board[0])
        result: List[str] = []

        # 1. Build the Trie
        root = TrieNode()
        for word in words:
            node = root
            for ch in word:
                if ch not in node.children:
                    node.children[ch] = TrieNode()
                node = node.children[ch]
            node.word = word

        # 2. DFS Backtracking
        def dfs(r: int, c: int, parent: TrieNode, ch: str) -> None:
            curr = parent.children.get(ch)
            if not curr:
                return

            # Match found
            if curr.word is not None:
                result.append(curr.word)
                curr.word = None  # Clear to avoid duplicates

            # Mark cell as visited
            board[r][c] = "#"

            # Explore neighbors
            for dr, dc in ((0, 1), (0, -1), (1, 0), (-1, 0)):
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols:
                    next_ch = board[nr][nc]
                    if next_ch != "#" and next_ch in curr.children:
                        dfs(nr, nc, curr, next_ch)

            # Backtrack
            board[r][c] = ch
```

```
            # 3. Trie Pruning: Remove leaf nodes
            if not curr.children:
                parent.children.pop(ch)

        # 3. Scan the grid
        for r in range(rows):
            for c in range(cols):
                ch = board[r][c]
                if ch in root.children:
                    dfs(r, c, root, ch)

        return result
```

19.4 Python-Specific Complexities & Implementation Considerations

19.4.1 1. Dynamic Garbage Collection & Pruning

- In Python, doing `parent.children.pop(ch)` cuts off reference paths to `curr`. Since Python uses reference counting combined with a cyclic garbage collector, removing this reference automatically schedules `curr` (and any unreferenced sub-children) for garbage collection. This provides painless dynamic memory management without manual cleanup.

19.4.2 2. Eliminating Result Sorting

- Some implementations sort the output array (`return sorted(result)`) before returning. This is a waste of CPU cycles; LeetCode accepts results in any order. For a result list containing thousands of matches, sorting adds unnecessary $\mathcal{O}(K \log K)$ overhead.

19.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(M \cdot N \cdot 4 \cdot 3^{L-1})$	Worst-case where we check all directions at each step. $M \times N$ is the grid size, and L is the maximum word length (up to 10). In practice, Trie pruning dramatically reduces actual operations to a small fraction of the worst-case.

Metric	Complexity	Description
Space Complexity	$\mathcal{O}(K \cdot L)$	Memory used by the Trie, where K is the number of words and L is the maximum word length. The stack frame depth for DFS takes $\mathcal{O}(L)$ auxiliary space.

19.6 Common Follow-Up Questions & How to Answer

19.6.1 Q1: Why not use a hash set of words and check all grid prefixes?

- **The Problem**: Searching prefixes in a Set of strings requires doing string slice checks. To check if `oat` is a prefix of any word in a Set, we must scan the set or pre-populate it with all prefix slices.
- **The Difference**: A Trie naturally nests common prefixes. Most importantly, a Set does not support **pruning**. If we find a word in a Set, we cannot easily mark a branch as dead and prevent future board scans from exploring it. A Trie allows immediate node deletions.

19.6.2 Q2: How would you handle this problem on a massive grid (e.g. 1000×1000)?

- **Grid Partitioning**: Divide the grid into overlapping sub-grids and search them in parallel using multiple threads.
- **Filtering Start Coordinates**: Pre-scan the grid to index start coordinates of matching Trie root characters. If root characters are sparse, we only invoke DFS on those specific coordinates instead of visiting every single cell in a double loop.

19.7 Pro-Tip: How to Impress the Interviewer

- **Implement Trie Pruning**: Writing the pruning step (`parent.children.pop(ch)` or `delete curr`) proves you design code with performance constraints in mind. It shows you understand tree restructuring on the fly to bound search complexity.
- **Leverage Heap Space Optimization (C++)**: Discussing `new` and `delete` allocation costs. Explain how dynamically allocating millions of tiny nodes causes memory fragmentation. Suggest pre-allocating an array of `TrieNode` structures (using a flat vector pool) to achieve contiguous memory layout, maximizing cash hits.

20 Concatenated Words

- **Category**: Coding
- **Difficulty**: Hard
- **Target Role**: Software Engineer / AI Systems Architect
- **Source**: LeetCode 472, Glassdoor
- **Flashcards**: Trie deck

20.1 Question Description

Given an array of strings `words` (without duplicates), return *all the* ***concatenated words*** *in the given list of words.*

A **concatenated word** is defined as a string that is comprised entirely of at least two shorter words (not necessarily distinct) in the given array.

20.1.1 Examples

- **Input**:

```
words = ["cat","cats","catsdogcats","dog","dogcatsdog","hippopotamuses","rat","ratcatdogcat"]
```

  - **Output**: `["catsdogcats","dogcatsdog","ratcatdogcat"]`
  - **Explanation**:
    - `"catsdogcats"` can be concatenated by `"cats"`, `"dog"` and `"cats"`.
    - `"dogcatsdog"` can be concatenated by `"dog"`, `"cats"` and `"dog"`.
    - `"ratcatdogcat"` can be concatenated by `"rat"`, `"cat"`, `"dog"` and `"cat"`.

- **Input**:

```
words = ["cat","dog","catdog"]
```

  - **Output**: `["catdog"]`

20.1.2 Constraints

- $1 \leq$ words.length $\leq 10^4$
- $1 \leq$ words$[i]$.length ≤ 30
- `words[i]` consists of only lowercase English letters.
- All strings in `words` are unique.
- $1 \leq \sum$ words$[i]$.length $\leq 10^5$

20.2 Detailed Solution (C++)

A naive search for partitions would result in exponential time complexity due to redundant path calculations. We can solve this efficiently using a combination of **Sorting**, **Trie**, and **DFS with Memoization**:

1. **Sort Words by Length**: We sort the words in ascending order of their length. By doing this, we guarantee that when we process a word, all potential component words (which must be strictly shorter) have already been analyzed and inserted into the Trie.
2. **DFS with Memoization**: For each word, we check if it can be formed by the shorter words currently in the Trie.
   - To prevent worst-case exponential backtracking (e.g., matching `aaaa...ab`), we maintain a `memo` array of size `L` where `memo[start]` caches whether the suffix starting at index `start` can be successfully concatenated.

3. **Insert into Trie**: After evaluating the word, we insert it into the Trie so it can be used to form even longer words.

20.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <string>
#include <algorithm>

class Solution {
private:
    struct TrieNode {
        TrieNode* children[26] = {nullptr};
        bool is_end = false;

        // Recursive destructor to prevent memory leaks
        ~TrieNode() {
            for (TrieNode* child : children) {
                delete child;
            }
        }
    };

    TrieNode* root;

    void insert(const std::string& word) {
        TrieNode* cur = root;
        for (char ch : word) {
            int idx = ch - 'a';
            if (!cur->children[idx]) {
                cur->children[idx] = new TrieNode();
            }
            cur = cur->children[idx];
        }
        cur->is_end = true;
    }

    // DFS with Memoization to check if a suffix can be formed from Trie words
    bool canConcatenate(const std::string& word, size_t start, std::vector<int>& memo) const noexcept {
        if (start == word.size()) {
            return true;
        }
        if (memo[start] != -1) {
            return memo[start];
```

```
        }

        TrieNode* cur = root;
        for (size_t i = start; i < word.size(); ++i) {
            int idx = word[i] - 'a';
            if (!cur->children[idx]) {
                break; // No matching prefix in Trie, prune branch
            }
            cur = cur->children[idx];
            if (cur->is_end) {
                if (canConcatenate(word, i + 1, memo)) {
                    return memo[start] = 1; // Cache success
                }
            }
        }
        return memo[start] = 0; // Cache failure
    }

public:
    Solution() : root(new TrieNode()) {}

    ~Solution() {
        delete root;
    }

    // Disable copy operations to maintain strict RAII safety
    Solution(const Solution&) = delete;
    Solution& operator=(const Solution&) = delete;

    std::vector<std::string> findAllConcatenatedWordsInADict(std::vector<std::string>& words) {
        // Sort words by length in ascending order
        std::sort(words.begin(), words.end(), [](const std::string& a, const std::string& b) {
            return a.size() < b.size();
        });

        std::vector<std::string> result;
        for (const std::string& word : words) {
            if (word.empty()) continue;

            // memo: -1 = unvisited, 0 = False, 1 = True
            std::vector<int> memo(word.size(), -1);
            if (canConcatenate(word, 0, memo)) {
                result.push_back(word);
            }
```

```
            // Always insert the word to allow it to form longer words
            insert(word);
        }
        return result;
    }
};
```

20.3 Detailed Solution (Python)

20.3.1 Standard Python Production Code

Below is the Python equivalent using `__slots__` for Trie memory efficiency and a simple list for fast state memoization.

```
from typing import List

class TrieNode:
    __slots__ = ("children", "is_end")

    def __init__(self) -> None:
        self.children: dict[str, TrieNode] = {}
        self.is_end: bool = False

class Solution:
    def findAllConcatenatedWordsInADict(self, words: List[str]) -> List[str]:
        # Sort by length: guarantees only shorter words exist in the Trie
        words.sort(key=len)

        root = TrieNode()

        def insert(word: str) -> None:
            node = root
            for ch in word:
                if ch not in node.children:
                    node.children[ch] = TrieNode()
                node = node.children[ch]
            node.is_end = True

        def can_concatenate(word: str, start: int, memo: List[int]) -> bool:
            if start == len(word):
                return True
            if memo[start] != -1:
                return memo[start] == 1
```

```
        node = root
        for i in range(start, len(word)):
            ch = word[i]
            if ch not in node.children:
                break
            node = node.children[ch]
            if node.is_end:
                if can_concatenate(word, i + 1, memo):
                    memo[start] = 1
                    return True

        memo[start] = 0
        return False

    result: List[str] = []
    for word in words:
        if not word:
            continue
        # Memo table initialized to -1 (unvisited)
        memo = [-1] * len(word)
        if can_concatenate(word, 0, memo):
            result.append(word)
        insert(word)

    return result
```

20.4 Python-Specific Complexities & Implementation Considerations

20.4.1 1. Memoization with Flat Lists vs `@lru_cache`

- Python's `@lru_cache` from `functools` is a convenient way to cache function results, but it introduces substantial lookup and function-call overhead.
- Since word lengths are small ($L \leq 30$), allocating a simple list `memo = [-1] * len(word)` is far faster. List indexing `memo[start]` runs at C-speed and avoids hashing arguments.

20.4.2 2. Recursion Limit

- Python's default recursion limit is 1000. Because the maximum length of a word is constrained to 30, our recursion depth will never exceed 30, making recursive DFS safe and free of `RecursionError` risks.

20.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(N \log N \cdot L + N \cdot L^2)$	N is the number of words, L is the maximum length of a word. Sorting takes $\mathcal{O}(N \log N \cdot L)$ time. For each word, memoized DFS visits at most L indices, performing at most L character lookups.
Space Complexity	$\mathcal{O}(N \cdot L)$	Memory used to store words in the Trie. The recursion stack and memo table require $\mathcal{O}(L)$ auxiliary space.

20.6 Common Follow-Up Questions & How to Answer

20.6.1 Q1: Can we solve this problem without sorting the words first?

- **The Answer**: Yes, we can. We can insert all words into a Trie or a Hash Set first.
- **The Catch**: During the search for `word`, we must ensure it doesn't match itself as a single piece (since a concatenated word requires at least two component words).
- To do this, we pass a `count` parameter in the DFS:

```
def dfs(word, start, count):
    # ...
    # At the end of the string, return True only if count >= 2
```

- **Comparison**: Sorting by length is cleaner because we only check if a word is formed by strictly shorter words, naturally preventing a word from matching itself.

20.6.2 Q2: How does a Hash Set approach compare to the Trie approach?

- **The Hash Set Solution**: We store all words in a Hash Set. To check if a word is concatenated, we run a DP/DFS similar to Word Break, slicing the string at different indices and looking up slices in the Set.
- **Trie Advantage**: A Trie is more efficient when words share prefixes because we scan character-by-character. If a prefix is not in the Trie, we break immediately. In contrast, checking slices in a Set (e.g. `word[start:i]`) forces us to copy sub-strings, incurring copying overhead.

20.7 Pro-Tip: How to Impress the Interviewer

- **Discuss DP Memoization explicitly**: Explain how checking `memo` avoids exponential branching on pathological inputs like `aaaa...aaaab`. Showing that you proactively identify exponential worst-cases

and apply memoization is a hallmark of a senior candidate.

- **Analyze String Copy Costs**: Point out that using a Trie to match characters sequentially avoids creating intermediate substring copies (which would occur if you used string slicing with a Hash Set). This highlights your awareness of garbage collection and allocation costs.